

Universidade do Minho

Resolução da Ficha do TP2
Package Sniffer

Trabalho Prático 2 – Grupo 43

Redes de Computadores

Membros do grupo:
Eduardo Luis Dias Pinto – a111523
Rui Miguel Silva Rebelo – a112411
Francisco Duarte Fernandes Silva – a100752

Universidade do Minho
2026
Licenciatura em Engenharia Informática

Índice

1 | Introdução 4
2 | Arquitetura do Sistema 5
2.1 | Metodologia de Análise Protocolar 5
2.2 | Visualização e Métricas de Captura 13
3 | Detalhes da Implementação 14
3.1 | Análise Detalhada do Fluxo de Execução (`main.py`) 14
3.1.1 | Lógica das Funcionalidades do Menu 14
3.1.2 | Mecanismos de Controlo e Segurança 15
3.1.3 | Lógica de Processamento em Tempo Real (Callback) 15
3.1.4 | Persistência de Dados e Logging 16
3.1.5 | Geração de Métricas Estatísticas 16
3.2 | Lógica de Processamento e Dissecação (`protocol.py`) 17
3.2.1 | Estratégia de Identificação de Protocolos 17
3.2.2 | Algoritmo de Reconstrução da Stack 17
3.2.3 | Extração Dinâmica de Campos por Camada 19
3.3 | Implementação do Motor de Captura (`sniffer.py`) 22
3.3.1 | A Classe `PacketSniffer` e o Método `start_capture` 22
3.3.2 | Interação com a Interface de Rede 23
3.4 | Gestão de Configuração e Recursos (`interface.py`) 23
3.4.1 | Recolha de Parâmetros (`get_config`) 23
3.4.2 | Enumeração de Hardware (`show_interfaces`) 24
4 | Parte A - Instanciação e captura na rede do emulador *CORE* 25
4.1 | Teste do programa no CORE: Topologia e Validação 25
4.1.1 | Topologia 25
4.1.2 | Testes de validação 26
5 | Parte B — Transposição para a interface real do PC 44
5.1 | Análise na Interface Real e Obstáculos Encontrados 44
5.1.1 | Transição do Ambiente CORE para a Interface Real 44
5.1.2 | Adaptação do Sniffer à Interface Real 44
5.1.3 | Protocolos Validados na Interface Real 45
5.1.4 | Testes de Validação na Interface Real 45
5.1.5 | Diferenças Observadas em Relação ao Ambiente CORE 59
5.1.6 | Obstáculos Técnicos Encontrados e Resolução 60
5.1.7 | Módulo de Injeção de Tráfego (gerador.py) 61

5.2 | Limitações da Aplicação 62
5.2.1 | Protocolos Não Suportados ou Parcialmente Analisados 62
5.2.2 | Limitações de Desempenho e Parsing 62
5.2.3 | Funcionalidades Previstas mas Não Implementadas 63
6 | Conclusão 64

1 | Introdução

No âmbito da unidade curricular de Redes de Computadores, este relatório documenta o desenvolvimento e a validação de uma ferramenta de monitorização e análise de tráfego, designada por **Packet Sniffer**. A análise passiva de redes constitui uma competência fundamental para a compreensão dos fluxos de comunicação, permitindo observar na prática a abstração teórica das camadas de rede e a interação entre diversos protocolos.

O objetivo principal deste projeto consistiu na criação de uma aplicação modular em *Python*, capaz de intercetar, dissecar e apresentar dados de tráfego de forma estruturada. Utilizando a biblioteca **Scapy**, o sistema permite a inspeção profunda de pacotes (**Deep Packet Inspection**), desde a camada de ligação de dados até à camada de aplicação.

A ferramenta foi submetida a dois cenários de teste distintos:

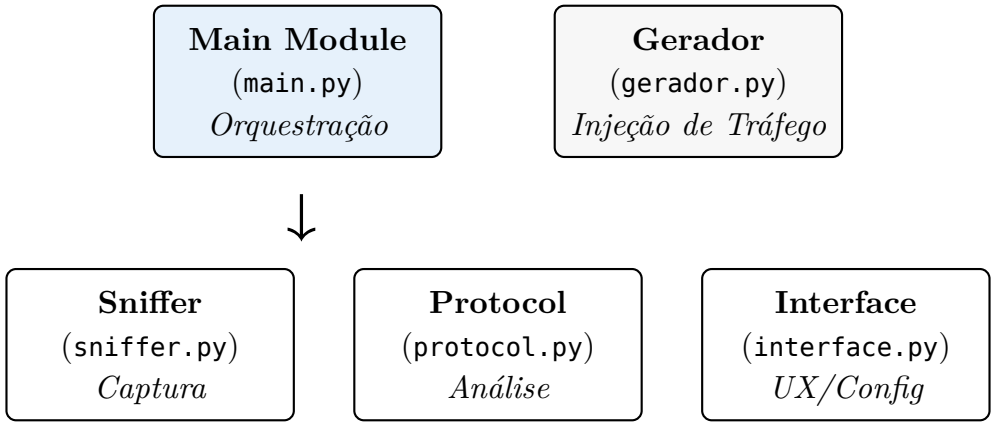
1. **Ambiente Simulado:** Utilização do emulador *CORE* para validar o comportamento do software numa topologia controlada, permitindo a análise de interações fundamentais como o protocolo ARP e o estabelecimento de ligações TCP.
2. **Ambiente Real:** Execução na interface de rede do sistema anfitrião, onde se explorou a captura de tráfego em redes Wi-Fi e Ethernet, enfrentando desafios como a densidade de tráfego, a grande complexidade na captura de tráfego de redes Wi-Fi e a diversidade de protocolos contemporâneos (ex: QUIC, IPv6).

Este documento detalha a arquitetura modular da aplicação, a metodologia utilizada para a interpretação de cada protocolo e os resultados obtidos durante as fases de experimentação.

2 | Arquitetura do Sistema

O software foi desenvolvido sob um paradigma de design modular, segmentando as responsabilidades de captura, processamento de dados e interface para garantir a escalabilidade e a manutenção do código.

- **Módulo de Captura** (`sniffer.py`): Implementa a lógica de baixo nível para a interceção de tráfego. Utiliza a biblioteca *Scapy* para interagir com o controlador da interface de rede, operando na captura de pacotes em bruto (**raw packets**) de acordo com os filtros *BPF definidos*(Pedidos ao utilizador).
- **Módulo de Dissecação** (`protocol.py`): Atua como o motor de análise protocolar. Este módulo processa o *payload* de cada pacote, identificando a **stack** de protocolos desde a Camada 2 até à Camada 7. É responsável pela tradução de valores hexadecimais e campos de cabeçalho em informação estruturada de forma a melhor acesso a dados fáceis de ler no ecrã pelo utilizador. Além disso também permite a observação de fragmentação de pacotes em IPv4.
- **Módulo de Interface e Orquestração** (`interface.py` & `main.py`): Gere o fluxo de execução e a interação com o utilizador. O `main.py` coordena o *callback* de captura em tempo real para além de definir a principal interface do programa, enquanto o `interface.py` isola a lógica de configuração e listagem de recursos de rede.
- **Módulo de Injeção de Tráfego** (`gerador.py`): Uma ferramenta utilitária concebida para a validação do sistema através da geração de pacotes sintéticos, permitindo testar a eficácia da dissecação em diversos cenários controlados. Vai ser referido este módulo nas últimas secções abaixo.



Listagem 1 : Hierarquia de ficheiros e organização modular do sistema

2.1 | Metodologia de Análise Protocolar

A análise segue o modelo de camadas (**OSI**), processando sequencialmente os cabeçalhos para reconstruir a hierarquia de comunicação, conforme ilustrado na Figura 1.

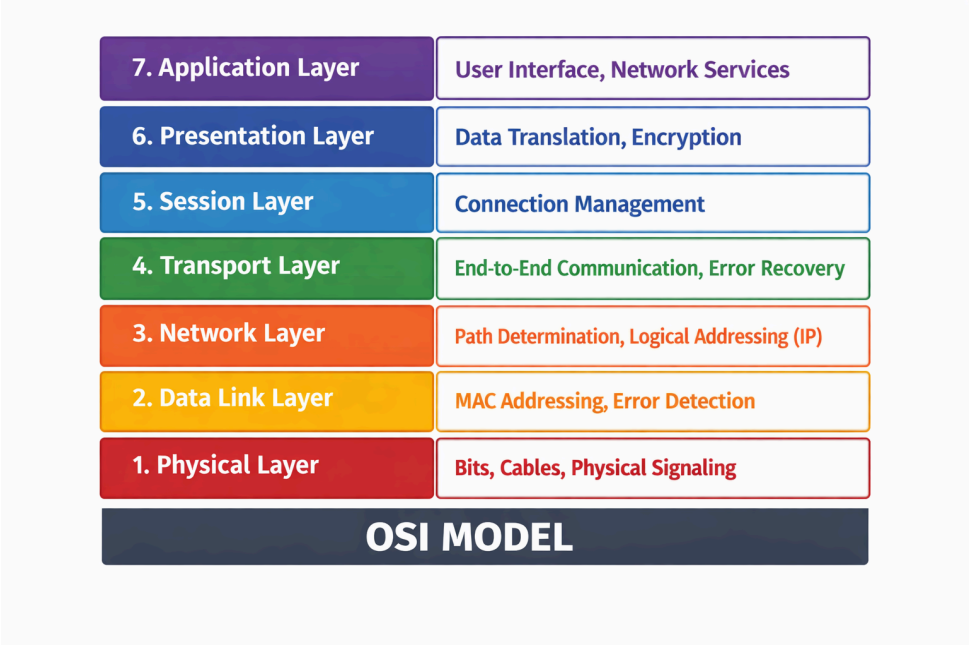


Figura 1 : Correspondência entre as camadas do Modelo OSI e os protocolos analisados pelo sistema.

1. **Ligação de Dados (L2):** Extração de endereços MAC através da camada `Ethernet`. O sistema identifica mensagens de controlo **ARP**, distinguindo entre pacotes de pedido (**Request** — "Who has") e resposta (**Reply** — "Is at") com base no campo de operação.
    - O **ARP (Address Resolution Protocol)** opera numa interface crítica entre a Camada de Rede (L3) e a Camada de Ligação de Dados (L2), sendo o protocolo de mapeamento fundamental entre o endereçamento lógico (IP) e o endereçamento físico (MAC) numa rede local. Na arquitetura TCP/IP, as aplicações comunicam através de endereços IP, mas, para que um *frame* seja entregue num segmento físico Ethernet, o hardware exige o endereço MAC do destino. O ARP resolve esta lacuna através de dois mecanismos: o **ARP Request**, onde um emissor envia um pacote em *broadcast* perguntando "Quem tem o IP X?", e o **ARP Reply**, onde o detentor do referido IP responde diretamente (*unicast*) com o seu endereço físico, permitindo a atualização da **ARP Cache**.

No contexto deste projeto, o sistema não se limita a detetar a passagem do pacote, mas reconstrói a interação de controlo. O *sniffer* analisa o campo de operação para distinguir pedidos (**Who has**) de respostas (**Is at**) e, ao extrair os IPs contidos no cabeçalho **ARP**, expõe o fluxo de resolução de vizinhança, tornando visível a topologia lógica da rede.

Curiosidade Técnica: O ARP baseia-se num princípio de "confiança implícita", o que significa que os dispositivos podem aceitar respostas mesmo sem terem feito um pedido prévio. Ao monitorizar visualmente a relação entre emissor e alvo, o nosso *sniffer* permite detetar anomalias, como tentativas de **ARP Spoofing**. Neste cenário, um atacante associa o seu endereço MAC ao IP do *Gateway*, permitindo-lhe intercetar ou manipular o tráfego da rede de forma silenciosa.

1. **Rede (L3):** Análise de pacotes **IPv4** e **IPv6**, com extração de endereços e metadados como o **TTL** (IPv4) ou **Hop Limit** (IPv6).
   - **IPv4 / IPv6:** Protocolos base de endereçamento que permitem rastrear a validade e a distância(número de saltos) lógica dos pacotes.
   - **Fragmentação IPv4:** O suporte à análise de fragmentação permite monitorizar campos críticos como o **ID**, **Fragment Offset** e **Flags** (MF/DF), sendo essencial para identificar se um pacote é o fragmento inicial, intermédio ou final.

     A figura abaixo apresenta uma comparação detalhada entre os cabeçalhos **IPv4** e **IPv6**, destacando a evolução e simplificação do protocolo:

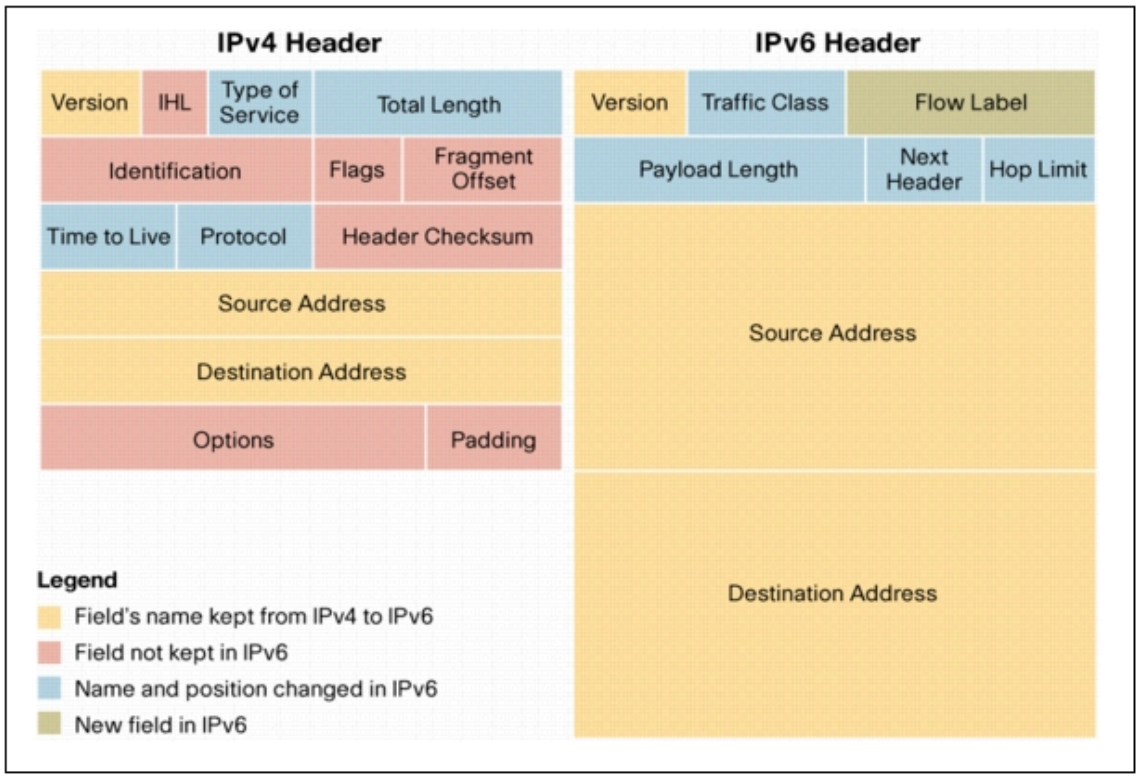


Figura 2 : Comparação estrutural entre cabeçalhos **IPv4** (esquerda) e **IPv6** (direita)

Conforme ilustrado, é possível identificar a transição entre versões através da legenda de cores:

- **Amarelo**: Campos que mantiveram o nome e a função.
- **Azul**: Campos que mudaram de nome ou posição (ex: o Protocol no IPv4 passou a Next Header no IPv6).
- **Vermelho**: Campos que foram removidos no IPv6 para simplificar o processamento nos routers (como o Checksum e os campos de fragmentação, que agora são tratados de forma diferente).
- **Verde**: Introdução do campo Flow Label, exclusivo do IPv6, utilizado para identificar fluxos de pacotes específicos que requerem tratamento especial.

- **ICMP / ICMPv6:** Utilizados para diagnóstico e mensagens de erro (ex: *Ping*). O sistema identifica o tipo de mensagem(*Request* ou *Reply*), facilitando a análise de conectividade.

1. **Transporte (L4):** Segmentação entre **TCP** e **UDP**.
   - **TCP (Transmission Control Protocol):** Protocolo orientado à ligação. Realiza-se a decodificação das *flags* (S, A, P, F, R ou junções das mesmas) para monitorizar o estado do aperto de mão (*handshake*) e o fluxo de dados, conforme detalhado na Tabela 1.

Tabela 1 : Mapeamento de Flags TCP no Scapy

Sigla	Nome da Flag	Descrição e Função Técnica
S	SYN	Sincronização de números de sequência (Handshake).
A	ACK	Confirmação de receção de pacotes anteriores.
P / PA	PSH	Instrução para envio imediato dos dados à aplicação.
F / FA	FIN	Finalização da transmissão de dados.
R	RST	Aborto imediato da ligação por erro ou porta fechada.
SA	SYN-ACK	Resposta combinada: aceitação e confirmação de ligação.

   - **UDP (User Datagram Protocol):** Protocolo sem ligação, focado na velocidade. O sistema mapeia as portas para identificar serviços de tempo real ou consultas rápidas.

Um exemplo da comparação entre o fluxo de comunicação de um pacote enviado por **TCP** e outro por **UDP** encontra-se demonstrado na figura abaixo:

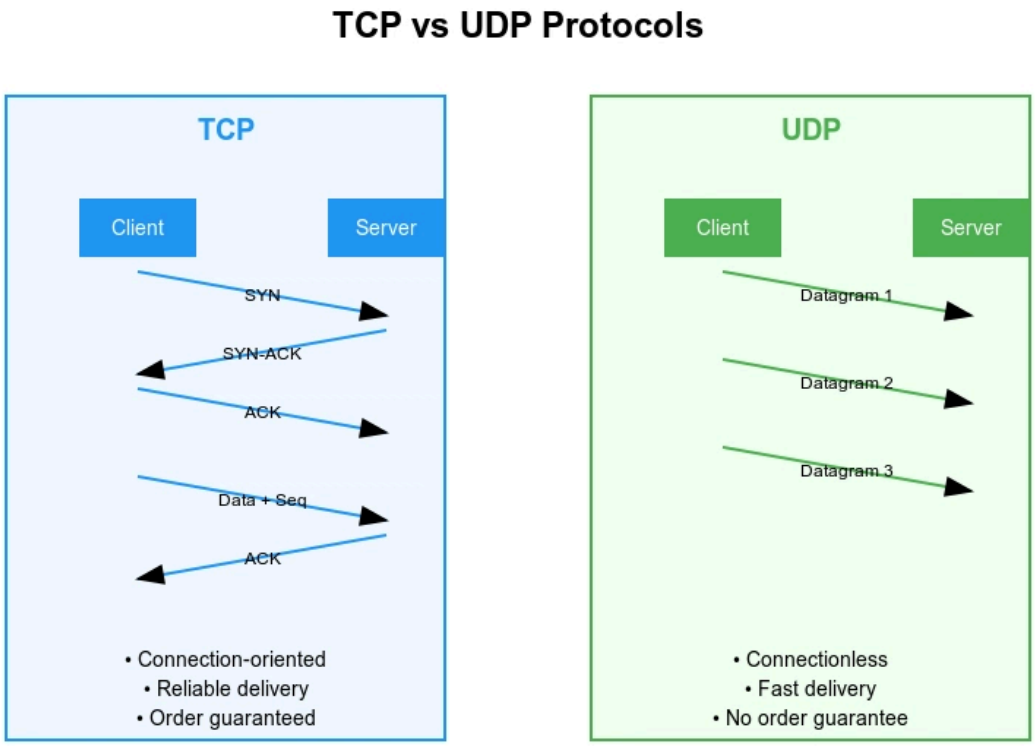


Figura 3 : Comparação entre o funcionamento dos protocolos **TCP** e **UDP**

Segundo a figura, o **TCP** estabelece a ligação através de um processo de três passos designado por ***Three-Way Handshake***:

1. O cliente envia um pacote com a **flag SYN (*Synchronize*)** para iniciar a ligação.
2. O servidor responde com as **flags SYN-ACK**, confirmando que recebeu o pedido e que está pronto para comunicar.
3. O cliente finaliza com a **flag ACK (*Acknowledge*)**, estabelecendo formalmente a ligação.

Só após esta fase é que os dados são efetivamente transferidos. O servidor confirma a receção de cada pacote de dados através de um **ACK**, garantindo que a informação chegou íntegra e na ordem correta.

Em contraste, o **UDP** é um protocolo muito mais simples e "sem ligação" (*connectionless*). O cliente envia os datagramas diretamente para o servidor sem qualquer verificação prévia ou confirmação de receção. Isto resulta numa comunicação significativamente mais rápida e com menos tráfego de controlo (*overhead*), mas não oferece garantias de que os dados cheguem ao destino ou que cheguem na sequência correta.

2. **Aplicação (L7):** Identificação de serviços através de mapeamento de portas e inspeção de *payload*:

   - **DNS - Domain Name System (UDP Porta 53):** Responsável pela resolução de nomes de domínio. O sistema realiza a inspeção do campo `qd.qname` para extrair o endereço solicitado (ex: *google.com* ou *googlecast.*tcp.local.).

     Utilidade e Uso no Sniffer: O DNS atua como o diretório da infraestrutura de rede. Ao capturar estas consultas, o *sniffer* consegue identificar as intenções de navegação do utilizador antes de qualquer transferência de dados ocorrer. Esta capacidade é crucial para a auditoria de rede, permitindo detetar pedidos a domínios maliciosos ou não autorizados de forma preventiva.

   - **DHCP(UDP Portas 67 e 68)/DHCPv6(UDP Portas 546 e 547) - Dynamic Host Configuration Protocol:** Responsável pela configuração dinâmica de endereços de rede. O *sniffer* identifica as fases do processo de atribuição: *Discover*, *Offer*, *Request* e *ACK*.

     Utilidade: Por operar sobre **UDP**, este protocolo permite que um dispositivo obtenha um endereço IP automaticamente ao ingressar na rede. A sua captura é fundamental para a gestão de segurança, pois permite detetar em tempo real a entrada de novos dispositivos na infraestrutura e analisar a distribuição e validade dos recursos de endereçamento atribuídos.

   - **HTTP - Hypertext Transfer Protocol (TCP Porta 80):** Protocolo de transferência de hipertexto que opera sobre **TCP**. O *sniffer* realiza a inspeção de conteúdo em bruto (**Raw**) para identificar métodos como `GET` ou `POST` e validar as versões do protocolo em uso.

     Curiosidade: Por ser um protocolo de texto simples, o HTTP permite a leitura integral do conteúdo (mensagens, formulários). Em contraste, o **HTTPS** utiliza cifragem(codificação) na camada de transporte, tornando o conteúdo privado e inacessível a ferramentas de captura passiva.

   - **Acesso Remoto:**

     - **SSH - Secure Shell (TCP Porta 22):** Este protocolo que opera em **TCP** identifica tráfego codificado.

Curiosidade: O **SSH** é o sucessor seguro do **Telnet**, onde mesmo que o *sniffer* capture o *payload*, o conteúdo será ilegível, ou seja, não é possível visualizar o seu conteúdo.

- **Telnet - Telecommunication Network (TCP Porta 23):** Este protocolo que opera em **TCP** identifica comunicações em texto limpo não protegido.

  Curiosidade: Telnet é um protocolo perigoso, pois permite capturar palavras-passe de administração diretamente no *payload* sendo o antecessor do **SSH**.

- **FTP - File Transfer Protocol (TCP Porta 21):** Este protocolo que opera em **TCP** deteta transferência de ficheiros. É um dos protocolos mais antigos da Internet, ainda utilizado para gestão de servidores.

- **Correio Eletrónico:**
  - **SMTP - Simple Mail Transfer Protocol (TCP Porta 25):** É o protocolo padrão da Internet que opera em **TCP** usado para enviar e retransmitir e-mails entre clientes de e-mail (como o Outlook ou o Gmail) e servidores de e-mail.
  - **POP3 - Post Office Protocol version 3 (TCP Porta 110):** É um protocolo que opera em **TCP** usado por clientes de e-mail (como o *Outlook* ou o *Thunderbird*) para transferência de e-mails de um servidor remoto para um único dispositivo local.

- **Sincronização e Web Moderna:**
  - **NTP - Network Time Protocol (UDP Porta 123):** É um protocolo que opera em **UDP** usado para sincronizar os relógios de computadores, servidores e dispositivos de rede com uma precisão de milissegundos em relação ao Tempo Universal Coordenado (UTC) em redes de dados com latência variável.

    Curiosidade: Sem este protocolo, a autenticação e os *logs* de rede falhariam devido a discrepâncias de tempo.

  - **SSDP - Simple Service Discovery Protocol (UDP Porta 1900)**: Protocolo que opera sobre UDP utilizado para a descoberta dinâmica de serviços e dispositivos (como *Smart TVs e impressoras*) em redes locais via *UPnP*(Universal Plug and Play).

O que é o UPnP? É uma tecnologia que permite que dispositivos (computadores, consolas, impressoras) se liguem e comuniquem numa rede de forma imediata, sem necessidade de configurações complicadas ou manuais.
Curiosidade: O SSDP utiliza mensagens de texto simples para anunciar serviços. Como não exige passwords ou autenticação para responder a pedidos de procura, é um dos protocolos mais vigiados em cibersegurança para evitar que aparelhos domésticos exponham informações da rede privada para o exterior.

- **HTTPS - Hypertext Transfer Protocol Secure (TCP Portas 443)/ QUIC - Quick UDP Internet Connections (UDP Portas 443):** O sniffer é capaz de identificar o tráfego Web moderno, distinguindo entre a pilha tradicional e a nova geração de protocolos.
  - **Distinção:** Enquanto o **HTTPS** convencional assenta sobre o protocolo **TCP** (que exige um processo de estabelecimento de ligação mais lento e rígido), o **QUIC** opera sobre **UDP**. Esta mudança permite ao **QUIC** gerir a retransmissão de dados de forma independente, evitando o "bloqueio de cabeça de linha" (**Head-of-line blocking**) comum no **TCP**.

    Curiosidade: O QUIC, desenvolvido inicialmente pela Google, introduz o conceito de *0-RTT (Zero Round Trip Time) Handshake*. Isto significa que, se um cliente já comunicou com o servidor anteriormente, pode enviar dados úteis logo no primeiro pacote, eliminando os múltiplos passos de "aperto de mão"(*Handshake*) que tornam as ligações HTTPS tradicionais mais lentas em redes com alta latência.

Nota: Se for necessário adicionar a captura de pacotes de protocolos ainda implementados com layer UDP ou TCP, só é necessário adicionar ao dicionário do respetivo protocolo.

```
# Dicionários globais para fácil alcance de dados
TCP_MAP = {80: "HTTP", 443: "HTTPS", 22: "SSH", 21: "FTP", 23: "Telnet",
25 : "SMTP", 110 : "POP3" } # porta: protocolo(relacionado com essa porta)
UDP_MAP = {53: "DNS", 67: "DHCP", 68: "DHCP", 546: "DHCPv6", 547: "DHCPv6",
443: "QUIC/HTTPS", 123 : "NTP", 1900 : "SSDP"}
```

2.2 | Visualização e Métricas de Captura

O sistema utiliza a biblioteca **Rich** para a apresentação de dados, implementando um **"Live Print"** que detalha a *stack* de cada pacote capturado em tempo real. Após a conclusão da sessão, é gerada uma tabela estatística que mostra as estatísticas de tráfego por protocolo, apresentando métricas de contagem absoluta, percentagem de pacotes e distribuição de carga em bytes.

3 | Detalhes da Implementação

A implementação técnica do sistema tira partido da flexibilidade da biblioteca **Scapy** e da capacidade de estilização da **Rich**. Abaixo, detalha-se o funcionamento de cada componente nuclear.

3.1 | Análise Detalhada do Fluxo de Execução (`main.py`)

O ficheiro `main.py` atua como o cérebro do sistema, implementando uma máquina de estados simples através de um ciclo infinito (`while True`) que gere o menu e as sessões de captura.

3.1.1 | Lógica das Funcionalidades do Menu

O menu principal foi desenhado para ser intuitivo, utilizando códigos de cores para distinguir ações de configuração (Ciano) de ações críticas ou de saída (Vermelho). A integração com o sistema operativo é feita de forma direta para maximizar a eficiência.

```
menu_content = (
            "[cyan][1][/cyan] Configure and Start Sniffer\n"
            "[cyan][2][/cyan] List of Available Interfaces\n"
            "[cyan][3][/cyan] Show Capture logs\n"
            "[cyan][4][/cyan] Clean Capture logs\n"
            "[cyan][5][/cyan] Clean Screen\n"
            f"[cyan][6][/cyan] Allow saving on the log.txt ({log_status})\n"
            "[red][7][/red] Exit"
        )
```

- **Configuração Dinâmica (Opção 1)**: Interage com o módulo `interface.py` para recolher parâmetros de filtragem. A flexibilidade do filtro **BPF** (Berkeley Packet Filter) é um ponto forte, pois permite que o utilizador utilize a mesma sintaxe como a ferramenta *Wireshark*. Este comando permite a iniciação de captura de pacotes seja ela infinita ou de um número de pacotes pré-definido, para além de poder escolher uma das interfaces disponíveis para captura.
- **Abstração do Sistema Operativo (Opções 3, 4 e 5)**: O software utiliza a biblioteca `os` para executar comandos nativos do sistema:
  - O comando `cat log.txt` permite a leitura rápida de dados persistidos(capturas anteriores guardadas que servem como um histórico).
  - O comando `rm log.txt` gere a limpeza do histórico de capturas.
  - O comando `clear` garante a higiene visual da interface, essencial durante capturas de alto volume.
- **Persistência Condicional (Opção 6)**: Implementa uma lógica de *toggle* (comutação). Esta funcionalidade é crucial para evitar o consumo desnecessário de armazenamento em disco quando o utilizador apenas deseja monitorizar tráfego em tempo real, sem necessidade de registo histórico.

- **Terminação Segura (Opção 7)**: Ao utilizar `sys.exit(0)`, o programa garante que todos os *buffers* de memória são libertados e o descritor do socket de rede é fechado corretamente pelo sistema operativo.

Nota de Implementação: A escolha de manter o `session_count` e o estado da variável `logs` como variáveis globais permite que o utilizador alterne configurações entre diferentes rondas de captura sem perder o contexto da sessão atual.

3.1.2 | Mecanismos de Controlo e Segurança

- **Validação de Privilégios**: O sistema utiliza `os.getuid() != 0` para garantir que o utilizador possui permissões de super-utilizador (**root - sudo**). Esta verificação é importante, uma vez que a abertura de sockets em modo promíscuo para captura de tráfego bruto é uma operação restrita ao nível do *kernel*.
- **Gestão de Erros e Interrupções**: A captura de pacotes está envolvida num bloco `try...except KeyboardInterrupt`. Isto permite que o utilizador interrompa uma captura infinita premindo `Ctrl+C` sem que o programa termine com erros de *exceptions*, garantindo que as estatísticas finais e os **logs** sejam processados e guardados corretamente.

3.1.3 | Lógica de Processamento em Tempo Real (Callback)

A função interna `live_print(pkt)` é o motor de captura em tempo real.

- **Diferenciação de Protocolos**: Através do método `.haslayer()` do *Scapy*, o sistema identifica a natureza do pacote e incrementa dois contadores distintos:
  - `stats`: Contador absoluto de pacotes por protocolo.
  - `statsSize`: Acumulador do tamanho total (em bytes) utilizando a função `len(pkt)`.

  Exemplo de uma tabela:

```
                    Capture Statistics
┏━━━━━━━━━━┳━━━━━━━┳━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━┓
┃Protocol  ┃ Count ┃ Percentage(By Qtd) ┃ Total Size ┃Percentage(By Size)┃
┡━━━━━━━━━━╇━━━━━━━╇━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━┩
│ TCP      │     8 │              53.3% │  930 bytes │              50.0% │
│ UDP      │     6 │              40.0% │  877 bytes │              47.1% │
│ Other    │     1 │               6.7% │   54 bytes │               2.9% │
└──────────┴───────┴────────────────────┴────────────┴────────────────────┘

[*] Finished. Total: 15
```

Com estes dados é possível observar o fluxo e os tipos de protocolos que são transmitidos na interface escolhida.

- **Human-Readable Interface**: O uso de `time.sleep(0.5)` é uma escolha de design deliberada para permitir que o utilizador consiga ler a **stack** de protocolos de cada pacote no terminal antes da chegada do próximo, evitando o "inundamento" visual do ecrã.

3.1.4 | Persistência de Dados e Logging

Foi implementado um sistema de *logging* condicional gerido pela variável `logs` de acordo com seguintes princípios:

- **Escrita Formatada**: Quando ativo, o sistema utiliza o modo de abertura `"a"` (**append**) para não sobrescrever capturas anteriores e escreve as capturas feitas.
- **Enriquecimento de Dados**: O *log* não guarda apenas o resumo básico do Scapy(`scapy.summary()`); é invocado a função `protocol.get_detailed_info(p)` para registar a análise profunda de cada camada, replicando a informação o que foi visto no ecrã no *log*.

3.1.5 | Geração de Métricas Estatísticas

Após a captura, o sistema realiza uma análise pós-processamento para gerar uma tabela comparativa.

- **Cálculos Dinâmicos**: O sistema calcula percentagens relativas tanto para o número de pacotes como para o volume de dados:

$$P_{\{qtd\}} = \frac{\text{count}}{\text{total_pkts}} \times 100$$

$$P_{\{size\}} = \frac{\text{size}}{\text{total_size}} \times 100$$

- **Filtragem de Ruído**: A tabela estatística utiliza uma lógica de filtragem que apenas exibe protocolos que registaram atividade (`if count > 0`), resultando numa interface limpa e focada no tráfego real da rede.

```
  # calcula o total apenas do que está nos dicionários
                total_pkts_visible = sum(stats.values())
                total_size_visible = sum(statsSize.values())

                for proto in stats.keys():
                    count = stats[proto]
                    size = statsSize.get(proto, 0)

                    if count > 0:
# Calcula as percentagens tendo em conta a quantidade e tamanho dos pacotes
capturados
                        pct_count = (count / total_pkts_visible) * 100
                        pct_size = (size / total_size_visible) * 100
```

```
                if total_size_visible > 0 else 0

                table.add_row(
                    proto,
                    str(count),
                    f"{pct_count:.1f}%",
                    f"{size} bytes",
                    f"{pct_size:.1f}%"
                )
        console.print(table)
```

Nota Técnica sobre o Código: O uso de `os.system('clear')` e da biblioteca `Rich` são utilizadas de forma a garantir uma melhor experiência para o Utilizador, transformando uma ferramenta de linha de comando técnica numa interface intuitiva e organizada.

3.2 | Lógica de Processamento e Dissecação (`protocol.py`)

O ficheiro `protocol.py` implementa a lógica de análise profunda do sistema. A sua função principal é transformar o objeto binário capturado pelo *Scapy* numa representação textual detalhada, hierárquica e de bom visual.

3.2.1 | Estratégia de Identificação de Protocolos

A identificação não se baseia apenas no cabeçalho de transporte, mas numa combinação de fatores:

- **Mapeamento Estático**: O código utiliza os dicionários `TCP_MAP` e `UDP_MAP` para uma identificação rápida baseada em portas conhecidas (ex: 80 para *HTTP*, 53 para *DNS*).
- **Heurística de Payload (DPI)**: Para o protocolo HTTP, o sistema implementa uma verificação adicional. Mesmo que a porta não seja a padrão, o código inspeciona os primeiros bytes do *payload* `Raw` em busca de métodos específicos:

```
if pkt.haslayer(scapy.Raw):
    payload = pkt[scapy.Raw].load.decode(errors='ignore')
    if any(kw in payload for kw in ["GET ", "POST ", "HTTP/1."]):
        app_proto = "HTTP"
```

3.2.2 | Algoritmo de Reconstrução da Stack

O desafio de apresentar uma **"Stack"** de protocolos (ex: Ethernet -> IP -> TCP -> HTTP) foi resolvido com um ciclo iterativo que percorre a estrutura de camadas do pacote:

1. Iteração de *Payload*: O ciclo `while temp_pkt` avança através de temp_pkt.payload, extraindo o nome de cada camada encontrada.

2. Filtragem de Ruído: Utiliza-se uma lista IGNORE_LAYERS (Padding, Trailer, Raw) para evitar que metadados técnicos sujem a visualização, focando apenas nos protocolos de comunicação.

3. Tratamento de Exceções de Identificação: Quando o *Scapy* rotula uma camada como *Raw* mas a nossa lógica de análise já identificou o protocolo (ex: HTTP) através da verificação da porta(ex: para HTTP porta 80), o código substitui o nome genérico pelo nome do protocolo detetado, garantindo uma **stack** precisa mesmo em pacotes fragmentados ou de controlo (como *ACKs* vazios) como é possível observar no código feito:

```
# 1. Identificação do Protocolo de Aplicação (para a Stack)
# Para o dicionário do TCP (TCP_MAP)
if pkt.haslayer(scapy.TCP):
    sport, dport = pkt[scapy.TCP].sport, pkt[scapy.TCP].dport # Vai ver as
portas de origem e destino
    app_proto = TCP_MAP.get(dport, TCP_MAP.get(sport, "TCP")) # Verifica se
existem no dicionário
    # Reforço HTTP
    if pkt.haslayer(scapy.Raw):
        payload = pkt[scapy.Raw].load.decode(errors='ignore')
        if any(kw in payload for kw in ["GET ", "POST ", "HTTP/1."]):
            app_proto = "HTTP"

# Para o dicionário do UDP (UDP_MAP)
elif pkt.haslayer(scapy.UDP):
    sport, dport = pkt[scapy.UDP].sport, pkt[scapy.UDP].dport # Vai ver as
portas de origem e destino
    app_proto = UDP_MAP.get(dport, UDP_MAP.get(sport, "UDP")) # Verifica se
existem no dicionário
    if pkt.haslayer(scapy.DNS): app_proto = "DNS" # Identificar DNS
    elif pkt.haslayer(scapy.DHCP) or pkt.haslayer(scapy.DHCP6):  #
Identificar DHCP (seja IPv4 ou IPv6)
        app_proto = "DHCPv6" if pkt.haslayer(scapy.DHCP6) else "DHCP"

# 2. Construção da Stack
stack = []
IGNORE_LAYERS = ["Padding", "Trailer", "Raw"] # Para ficar com a stack mais
limpa
temp_pkt = pkt
while temp_pkt:
    name = temp_pkt.name
    if name == "Raw" and app_proto and app_proto not in ["TCP", "UDP"]: #
Se for de uma layer diferente o scapy não permite identificar o
protocolo(escreve RAW)
        stack.append(app_proto) # Se esse protocolo existir e não for TCP
```

```
ou UDP, escreve o nome acima identificado
    elif (name not in IGNORE_LAYERS): # Se não for alguns dos supostos
layers para ignorar ele acrescenta à stack o nome do protocolo
            stack.append(name)

    if not temp_pkt.payload and app_proto and app_proto not in ["TCP",
"UDP"]: # Garante que protocolos detetados sem Payload também aparecem na
stack (ex: ACKs de HTTP)
        if app_proto not in stack:
            stack.append(app_proto)

    temp_pkt = temp_pkt.payload
```

3.2.3 | Extração Dinâmica de Campos por Camada

A função *get_detailed_info* segmenta a extração por camadas **OSI**, demonstrando a versatilidade do *sniffer*:

1. ***Link Layer* (L2):** Inicialmente é detetado se o protocolo possui um cabeçalho *Ethernet*(**Nota**: A maior parte dos pacotes possui) garantido, que exista os MAC de destino e origem:

```
# --- Camada 2: Data Link Ethernet ---
if pkt.haslayer(scapy.Ether):
details.append(f"MAC Origin: {pkt[scapy.Ether].src}")
details.append(f"MAC Destination: {pkt[scapy.Ether].dst}")
```

2. **Camada de Rede (L3):** O sistema distingue fluxos IPv4 e IPv6. No IPv4, é extraído o campo TTL (Time to Live), enquanto no IPv6 obtém-se o Hop Limit, respeitando as nomenclaturas específicas de cada protocolo. Adicionalmente, implementou-se o suporte à análise de fragmentação IPv4, permitindo a monitorização de campos críticos como o **ID** (Identificação), as **Flags** (*MF - More Fragments* e *DF - Don't Fragment*) e o **Fragment Offset**, essencial para a reconstrução lógica de pacotes segmentados para além de indicar se é o primeiro, último ou intermédio pacote na fragmentação.

```
# --- Camada 3: Rede IP, IPv6 ou ARP ---
is_frag = False
if pkt.haslayer(scapy.IP):
    ip_layer = pkt[scapy.IP]
    details.append(f"IP Origin: {ip_layer.src}")
    details.append(f"IP Destination: {ip_layer.dst}")
    details.append(f"TTL: {ip_layer.ttl}")

    # --- Detalhes de Fragmentação ---
    # id: Identificador único para o conjunto de fragmentos
    # flags: MF (More Fragments), DF (Don't Fragment)
    # frag: Offset (posição do fragmento no pacote original)
```

```
        frag_id = ip_layer.id
        frag_offset = ip_layer.frag  # Este valor deve ser multiplicado por 8
    para obter o offset em bytes
        flags = ip_layer.flags
        status = "---"

        # Tradução das Flags para algo que se entenda
        flag_list = []
        if flags & 0x02:
            flag_list.append("(Don't Fragment)")
        if flags & 0x01:
            flag_list.append("(More Fragments)") # Se tiver a flag MF, é
    indicado que é fragmentado(is_frag)
            status = "Middle Fragment" if frag_offset > 0 else "First Fragment"
    # verifica se é o primeiro pacote ou intermédio
            is_frag = True
        elif frag_offset > 0:
            status = "Last Fragment" # Só pode ser o último pacote fragmentado
            is_frag = True
        if flags & 0x00: flag_list.append("(Reserved)") # Esta flag não possui
    muita importância(normalmente 0)

        flags_str = ", ".join(flag_list) if flag_list else "None"
        if is_frag:
            details.append(f"Fragment ID: {frag_id}")
            details.append(f"Fragment Offset: {frag_offset} ({frag_offset * 8}
    bytes)")
            details.append(f"Frag status: {status}")
            details.append(f"IP Flags: {flags} {flags_str}")

            if frag_offset > 0:
                app_proto = f"IP fragment (ID : {frag_id})"

        else: details.append("Not Fragmented")

    elif pkt.haslayer(scapy.IPv6):
        details.append(f"IPv6 Origin: {pkt[scapy.IPv6].src}")
        details.append(f"IPv6 Destination: {pkt[scapy.IPv6].dst}")
        details.append(f"Hop Limit: {pkt[scapy.IPv6].hlim}")
```

3. **Lógica de Controlo do protocolo ARP**: Em vez de apenas mostrar endereços, o analisador interpreta o campo de operação (*op*) dado pelo *Scapy* para traduzir códigos numéricos em descrições humanas: 1 para *Request*(**Who has**) e 2 para *Reply*(**It at**).

```
elif pkt.haslayer(scapy.ARP):
details.append(f"Protocol: ARP ({'Request (Who has)' if pkt[scapy.ARP].op == 1
else 'Reply (It at)'})")
details.append(f"Query: {pkt[scapy.ARP].psrc} -> {pkt[scapy.ARP].pdst}")
```

4. Dissecação de ICMP/ICMPv6: A função lida com a variação de tipos entre versões. Por exemplo, identifica que o tipo 128 no ICMPv6 equivale ao *"Echo Request"*, mantendo a consistência visual para o utilizador final, independentemente da versão do protocolo IP.

```
# --- Camada 3 Rede (raramente camada 4 -> Transporte): ICMP / ICMPv6 ---
elif pkt.haslayer(scapy.ICMP) or pkt.haslayer(scapy.ICMPv6EchoRequest) or
pkt.haslayer(scapy.ICMPv6EchoReply):
    is_v6 = pkt.haslayer(scapy.ICMPv6EchoRequest) or
pkt.haslayer(scapy.ICMPv6EchoReply) # Verifica de é IPv6
    proto_name = "ICMPv6" if is_v6 else "ICMP"

    if is_v6:
        # No IPv6, extraímos o tipo da camada que estiver presente(está divida
no scapy por ICMPv6EchoRequest e ICMPv6EchoReply)
        if pkt.haslayer(scapy.ICMPv6EchoRequest):
            tipo = pkt[scapy.ICMPv6EchoRequest].type
        else:
            tipo = pkt[scapy.ICMPv6EchoReply].type
        # Se for o tipo for 128 é Request, 129 é Reply
        msg = "Echo Request" if tipo == 128 else "Echo Reply" if tipo == 129
else f"Type {tipo}"
    else:
        # No IPv4 (ICMP normal)
        tipo = pkt[scapy.ICMP].type
        # Se for o tipo for 8 é Request, 0 é Reply
        msg = "Echo Request" if tipo == 8 else "Echo Reply" if tipo == 0 else
"Time Exceeded" if tipo == 11 else f"Type {tipo}"
    details.append(f"Protocol: {proto_name} ({msg})")
```

5. Parsing de Opções **DHCP**: No caso do *DHCP*, o código itera sobre a lista de opções para encontrar o *message-type*, mapeando os códigos (1, 2, 3, 5) para os estados do processo de atribuição de IP (*Discover, Offer, Request, ACK*).

```
# -- DHCP (camada 7 -> Aplicação) ---
elif pkt.haslayer(scapy.DHCP):
    options = pkt[scapy.DHCP].options
    msg_type = next((opt[1] for opt in options if isinstance(opt, tuple) and
opt[0] == 'message-type'), "Unknown")
    dhcp_msgs = {1: "Discover", 2: "Offer", 3: "Request", 5: "ACK"}
    details.append(f"DHCP Message: {dhcp_msgs.get(msg_type, msg_type)}")
```

Nota: O uso de `decode(errors='ignore')` na extração de dados Raw e DNS é fundamental para evitar que o programa falhe ao processar caracteres não-ASCII ou binários, garantindo que o *sniffer* continue operacional mesmo em condições de rede adversas.

Nota de implementação: O grupo analisou a possibilidade de capturar *frames* de Camada 2 do tipo **IEEE 802.11** (*Wi-Fi*). No entanto, verificou-se que, em modo gerido (*Managed Mode* é padrão), o Sistema Operativo e os controladores da placa de rede abstraem o tráfego rádio, encapsulando-o em cabeçalhos Ethernet II fictícios (processo conhecido como **Ethernet Emulation**). Para capturar os cabeçalhos reais de *Wi-Fi* (como *frames* de gestão **Beacon** ou **Probe**), seria necessário colocar a interface em Modo Monitor, o que apresenta dependências de hardware e permissões de sistema complexas. Assim, o grupo optou por focar o desenvolvimento na análise de *frames* Ethernet, garantindo a estabilidade do *sniffer* em ambientes de rede convencionais.

Visualização de um pacote "mascarado" pelo *Sistema Operativo* capturado pelo *sniffer*

```
┌──────── Packet #57 - 15:12:09 ────────┐
│ Stack: Ethernet                       │
│ ------------------------------        │
│ MAC Origin: 00:00:80:00:00:00         │
│ MAC Destination: 00:00:08:00:00:00    │
│ Protocol: Other                       │
│ ------------------------------        │
│ Total Size: 59 bytes                  │
└───────────────────────────────────────┘
```

3.3 | Implementação do Motor de Captura (`sniffer.py`)

O módulo `sniffer.py` atua como a camada de abstração de hardware do sistema. Em vez de espalhar a lógica complexa de captura pelo programa, o software utiliza a classe `PacketSniffer` para isolar as chamadas à biblioteca **Scapy**.

3.3.1 | A Classe `PacketSniffer` e o Método `start_capture`

O coração deste módulo é o método `scapy.sniff()`, que configura o socket de rede para o modo promíscuo. A implementação destaca-se pela utilização estratégica de três parâmetros fundamentais:

1. **Filtragem BPF (Berkeley Packet Filter)**: O parâmetro `filter` permite que o utilizador passe expressões como `tcp port 80` ou `icmp`.
   - **Vantagem**: A filtragem ocorre ao nível do *kernel*. Isto significa que o processador não gasta ciclos a enviar pacotes irrelevantes para a aplicação Python, aumentando drasticamente a performance em redes de alto débito.
2. **Mecanismo de Callback (`prn`)**: O parâmetro `prn` recebe a função `live_print` (definida no `main.py`).
   - **Processamento Assíncrono**: Sempre que a placa de rede recebe um pacote que corresponde ao filtro, o *Scapy* interrompe o fluxo principal para executar

esta função. Isto permite que o sistema exiba detalhes do pacote no ecrã enquanto a captura continua em segundo plano.

3. **Persistência em Memória (`store=True`)**: Ao contrário de *sniffers* que apenas exibem dados, este sistema armazena todos os pacotes capturados numa lista.
    - **Análise Pós-Captura**: Esta escolha permite que, após o encerramento da captura (seja por limite de pacotes ou interrupção do utilizador), o sistema possa calcular estatísticas precisas (como a soma de bytes totais) e gravar os resultados finais no ficheiro `log.txt`.

Configuração da Captura em `sniffer.py`

```
def start_capture(self, callback_func, bpf_filter="", count=10):
    packets = scapy.sniff(
        iface=self.interface,
        filter=bpf_filter, # Filtro BPF para eficiência
        store=True,        # Armazena para estatísticas finais
        count=count,       # Limite definido pelo utilizador
        prn=callback_func  # Executa live_print para cada pacote
    )
    return packets
```

3.3.2 | Interação com a Interface de Rede

O construtor **`__init__`** recebe o nome da interface (ex: eth0 ou wlp3s0). Caso o utilizador deixe este campo vazio, o *Scapy* seleciona automaticamente a interface padrão do sistema(normalmente a placa de rede física) com rota para a internet, garantindo que o software funcione mesmo que o utilizador não conheça tecnicamente os nomes das suas interfaces de rede.

3.4 | Gestão de Configuração e Recursos (`interface.py`)

O módulo `interface.py` é responsável pela camada de interação preliminar, garantindo que o utilizador fornece os parâmetros necessários para uma captura bem-sucedida sem sobrecarregar o fluxo principal do programa e a demonstração das interfaces disponíveis para captura.

3.4.1 | Recolha de Parâmetros (`get_config`)

A função `get_config` atua como um assistente de configuração. A sua implementação foca-se na flexibilidade dos inputs:

- **Tratamento de Inputs Omitidos**: No campo do limite de pacotes, foi utilizado no código a lógica `int(limit_input) if limit_input.strip() else 0`. Isto permite que, se o utilizador simplesmente pressionar *"Enter"*, o sistema assuma o valor `0` (captura infinita), evitando erros de conversão de tipos.

- **Abstração de Filtros**: Ao solicitar um filtro **BPF**, o módulo permite que o utilizador aplique lógica booleana complexa (ex: `tcp and port 443`) diretamente no motor de captura do *Scapy*, proporcionando uma experiência semelhante a ferramentas como o *Wireshark*.

3.4.2 | Enumeração de Hardware (`show_interfaces`)

Para facilitar a escolha do utilizador, o sistema comunica diretamente com o sistema operativo para listar as interfaces de rede ativas.

- **Lista Automática**: Através de `scapy.get_if_list()`, o software identifica todas as placas de rede (físicas e virtuais) disponíveis no *host*.
- **Apresentação Tabular**: Em vez de uma lista de texto bruto, os dados são injetados num objeto `Table` da biblioteca **Rich**. O uso de índices numerados (`enumerate(interfaces)`) permite que o utilizador identifique rapidamente o nome técnico da interface (como `wlp3s0` ou `lo`) que deve inserir na configuração.

Construção da Tabela de Interfaces em `interface.py`

```
def show_interfaces():
    # Mostra as interfaces disponíveis numa tabela organizada.
    interfaces = scapy.get_if_list()

    # Criar uma tabela estilizada(Coisas da lib rich)
    table = Table(
        title="[bold cyan]Available Network Interfaces[/bold cyan]",
        header_style="bold white",
        border_style="cyan"
    )

    table.add_column("ID", justify="center", style="yellow")
    table.add_column("Interface Name", style="blue")

    for i, iface in enumerate(interfaces):
        table.add_row(str(i + 1), iface)

    # Imprime a tabela no ecrã
    console.print(table)
```

4 | Parte A - Instanciação e captura na rede do emulador *CORE*

Q: Criar uma topologia de rede no emulador *CORE* que permita demonstrar os protocolos que desejam abordar.

4.1 | Teste do programa no CORE: Topologia e Validação

4.1.1 | Topologia

Com o intuito de testar o funcionamento do *Sniffer* num ambiente controlado, foi criada uma topologia no emulador **CORE** que consiste em dois *hosts*, **Cliente** e **Observador**, ligados através de um **Switch**. Este, por sua vez, conecta-se a um **Router** que estabelece a ligação a uma rede externa onde reside o Servidor. O nó **Observador** será o ponto central da experiência, onde o software desenvolvido será executado para capturar o tráfego circulante.

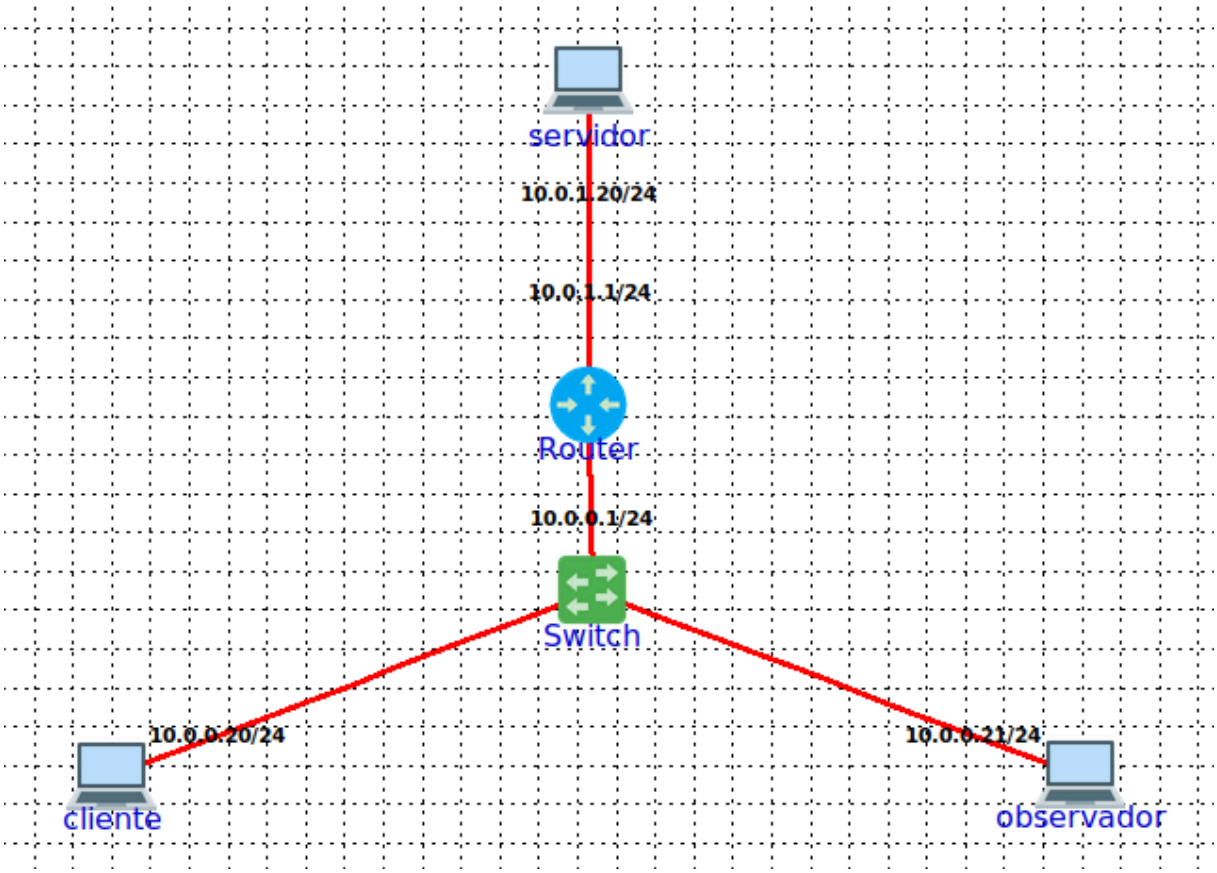


Figura 4 : Topologia definida no **CORE** usada pelo Grupo

Esta topologia permitirá testar o correto funcionamento do *sniffer* em todos os níveis da sua implementação: Local (*Layer 2*), Lógica (*Layer 3*), Fim-a-fim (*Layer 4*) e Aplicacional (*Layer 7*).

4.1.2 | Testes de validação

4.1.2.1 | Camada de Ligação e Rede (L2/L3)

Para começar, testou-se a capacidade do sniffer em capturar e analisar protocolos de camadas inferiores. Esta análise permite validar o comportamento da ferramenta com protocolos de controlo essenciais, que servem de base para toda a comunicação na rede.

4.1.2.1.1 | ARP (Address Resolution Protocol)

Na topologia definida no CORE, foi realizado um `ping` do **Cliente** para o **Observador**. Para forçar a geração de tráfego **ARP**, limpou-se previamente a cache do **Cliente** através do comando `ip neigh flush all`.

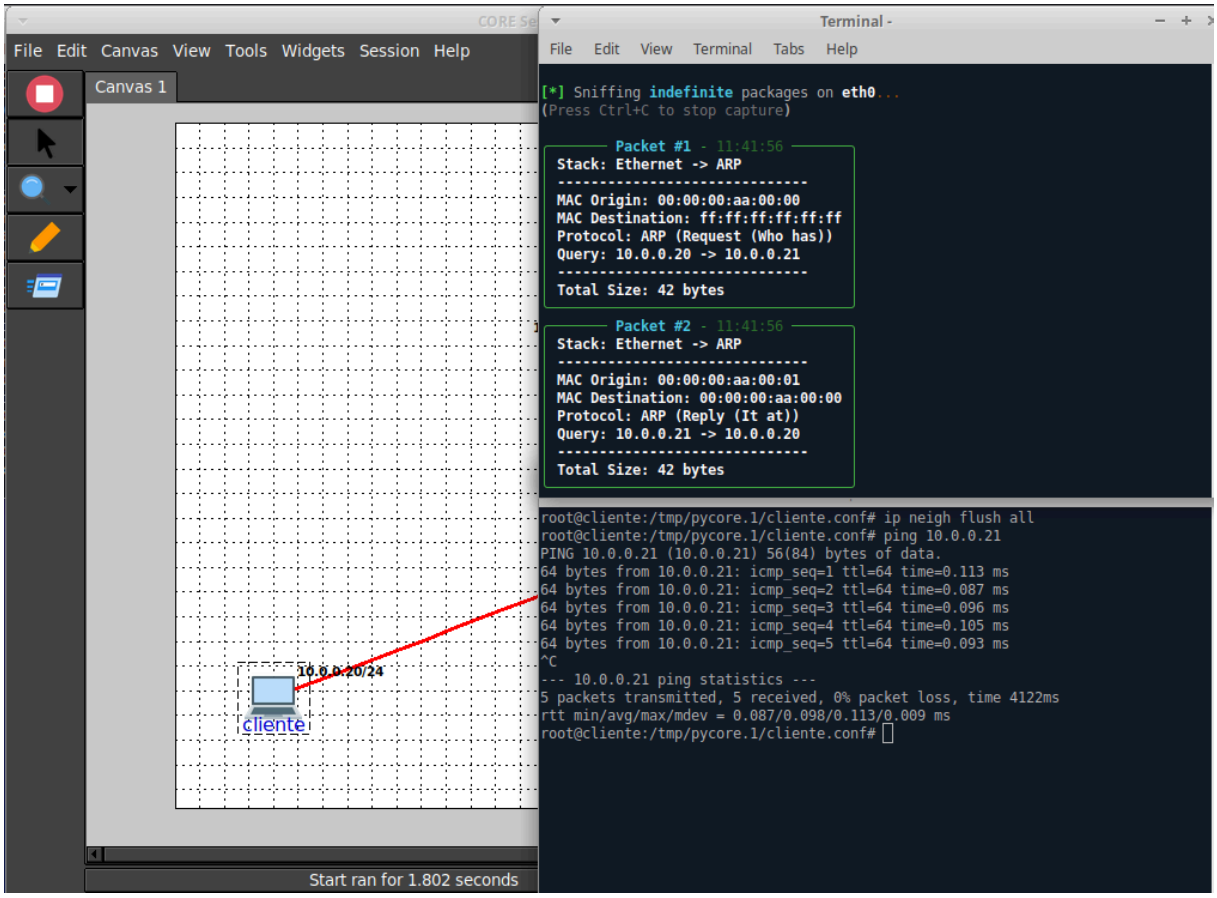


Figura 5 : Realização de um `ping` de **Cliente**(terminal debaixo) para **Observador**(terminal de cima). No terminal superior são visíveis os **ARP Requests** (`Who has?`) e os **ARP Replies** (`Is at`).

Este teste comprova que o *sniffer* não só identifica a presença de pacotes **ARP**, como também os interpreta corretamente. A ferramenta consegue extrair e exibir os endereços IP envolvidos na interrogação, distinguindo claramente as fases de pedido e resposta do protocolo.

4.1.2.1.2 | ICMP (Internet Control Message Protocol)

O **ICMP**, operando sobre IP, permite testar a conectividade e o percurso até ao **Servidor** através do **Router**. Este teste é fundamental para validar a identificação de mensagens de controlo e analisar a extração de campos dinâmicos do cabeçalho IP, como o **TTL** (Time to Live) e as *flags* de fragmentação.

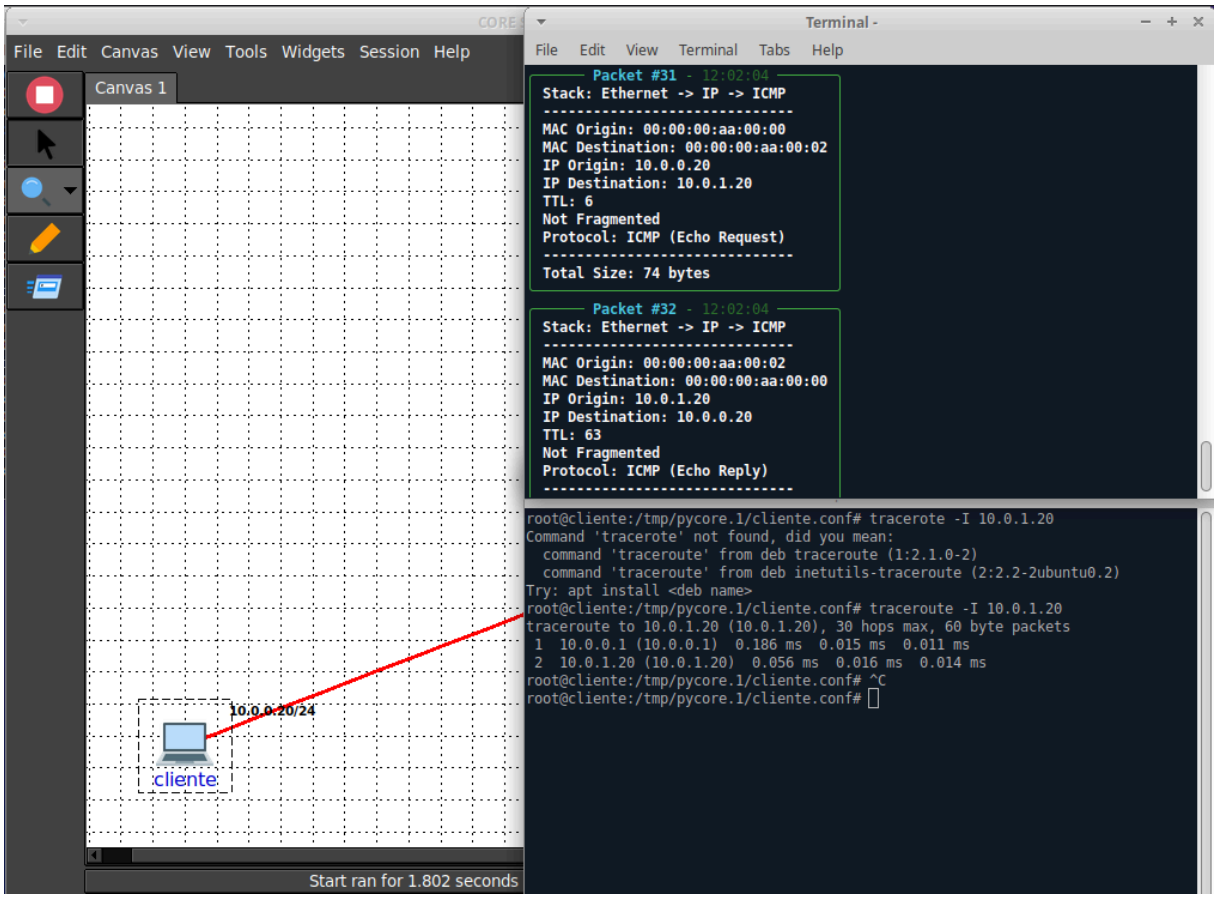


Figura 6 : Teste da implementação de **ICMP** através do comando `traceroute -I` de **Cliente** para o **Servidor**

Na Figura 6, observa-se a execução de um `traceroute -I` direcionado ao **Servidor**. O *sniffer* identifica corretamente o fluxo de **Echo Requests** e os respetivos **Echo Replies**. Além disso, através da análise detalhada do ficheiro de registo (log.txt), é possível verificar o incremento gradual do campo **TTL**: os primeiros pacotes, enviados com **TTL = 1**, são descartados pelos nós intermédios, gerando mensagens de erro **ICMP Time Exceeded**, também capturadas pela ferramenta:

```
--- Capture Logs ---

==================== SNIFFER NEW SESSION START ====================

------------------------------05/05/2026 12:09:18
```

```
--- Capture 1 ---
[1] [05/05/2026 12:09:21]
Ether / IP / ICMP 10.0.0.20 > 10.0.1.20 echo-request 0 / Raw

DETAILED PACKET INFO

Stack: Ethernet -> IP -> ICMP
------------------------------
MAC Origin: 00:00:00:aa:00:00
MAC Destination: 00:00:00:aa:00:02
IP Origin: 10.0.0.20
IP Destination: 10.0.1.20
TTL: 1
Not Fragmented
Protocol: ICMP (Echo Request)
------------------------------
Total Size: 74 bytes
------------------------------------------------------
[2] [05/05/2026 12:09:21]
Ether / IP / ICMP 10.0.0.1 > 10.0.0.20 time-exceeded ttl-zero-during-
transit / IPerror / ICMPerror / Raw

DETAILED PACKET INFO

Stack: Ethernet -> IP -> ICMP -> IP in ICMP -> ICMP in ICMP
------------------------------
MAC Origin: 00:00:00:aa:00:02
MAC Destination: 00:00:00:aa:00:00
IP Origin: 10.0.0.1
IP Destination: 10.0.0.20
TTL: 64
Not Fragmented
Protocol: ICMP (Time Exceeded)
------------------------------
Total Size: 102 bytes
------------------------------------------------------
[3] [05/05/2026 12:09:21]
Ether / IP / ICMP 10.0.0.20 > 10.0.1.20 echo-request 0 / Raw

DETAILED PACKET INFO

Stack: Ethernet -> IP -> ICMP
------------------------------
MAC Origin: 00:00:00:aa:00:00
MAC Destination: 00:00:00:aa:00:02
IP Origin: 10.0.0.20
IP Destination: 10.0.1.20
```

```
TTL: 1
Not Fragmented
Protocol: ICMP (Echo Request)
------------------------------
Total Size: 74 bytes
------------------------------------------------------
[4] [05/05/2026 12:09:21]
Ether / IP / ICMP 10.0.0.1 > 10.0.0.20 time-exceeded ttl-zero-during-
transit / IPerror / ICMPerror / Raw

DETAILED PACKET INFO

Stack: Ethernet -> IP -> ICMP -> IP in ICMP -> ICMP in ICMP
------------------------------
MAC Origin: 00:00:00:aa:00:02
MAC Destination: 00:00:00:aa:00:00
IP Origin: 10.0.0.1
IP Destination: 10.0.0.20
TTL: 64
Not Fragmented
Protocol: ICMP (Time Exceeded)
------------------------------
Total Size: 102 bytes
------------------------------------------------------
[5] [05/05/2026 12:09:21]
Ether / IP / ICMP 10.0.0.20 > 10.0.1.20 echo-request 0 / Raw

DETAILED PACKET INFO

Stack: Ethernet -> IP -> ICMP
------------------------------
MAC Origin: 00:00:00:aa:00:00
MAC Destination: 00:00:00:aa:00:02
IP Origin: 10.0.0.20
IP Destination: 10.0.1.20
TTL: 1
Not Fragmented
Protocol: ICMP (Echo Request)
------------------------------
Total Size: 74 bytes
------------------------------------------------------
[6] [05/05/2026 12:09:21]
Ether / IP / ICMP 10.0.0.1 > 10.0.0.20 time-exceeded ttl-zero-during-
transit / IPerror / ICMPerror / Raw

DETAILED PACKET INFO

Stack: Ethernet -> IP -> ICMP -> IP in ICMP -> ICMP in ICMP
```

```
------------------------------
MAC Origin: 00:00:00:aa:00:02
MAC Destination: 00:00:00:aa:00:00
IP Origin: 10.0.0.1
IP Destination: 10.0.0.20
TTL: 64
Not Fragmented
Protocol: ICMP (Time Exceeded)
------------------------------
Total Size: 102 bytes
------------------------------------------------------------
[7] [05/05/2026 12:09:21]
Ether / IP / ICMP 10.0.0.20 > 10.0.1.20 echo-request 0 / Raw

DETAILED PACKET INFO

Stack: Ethernet -> IP -> ICMP
------------------------------
MAC Origin: 00:00:00:aa:00:00
MAC Destination: 00:00:00:aa:00:02
IP Origin: 10.0.0.20
IP Destination: 10.0.1.20
TTL: 2
Not Fragmented
Protocol: ICMP (Echo Request)
------------------------------
Total Size: 74 bytes
------------------------------------------------------------
[8] [05/05/2026 12:09:21]
Ether / IP / ICMP 10.0.1.20 > 10.0.0.20 echo-reply 0 / Raw

DETAILED PACKET INFO

Stack: Ethernet -> IP -> ICMP
------------------------------
MAC Origin: 00:00:00:aa:00:02
MAC Destination: 00:00:00:aa:00:00
IP Origin: 10.0.1.20
IP Destination: 10.0.0.20
TTL: 63
Not Fragmented
Protocol: ICMP (Echo Reply)
------------------------------
Total Size: 74 bytes
------------------------------------------------------------
[9] [05/05/2026 12:09:21]
Ether / IP / ICMP 10.0.0.20 > 10.0.1.20 echo-request 0 / Raw
```

```
DETAILED PACKET INFO

Stack: Ethernet -> IP -> ICMP
------------------------------
MAC Origin: 00:00:00:aa:00:00
MAC Destination: 00:00:00:aa:00:02
IP Origin: 10.0.0.20
IP Destination: 10.0.1.20
TTL: 2
Not Fragmented
Protocol: ICMP (Echo Request)
------------------------------
Total Size: 74 bytes
------------------------------------------------------
[10] [05/05/2026 12:09:21]
Ether / IP / ICMP 10.0.1.20 > 10.0.0.20 echo-reply 0 / Raw

DETAILED PACKET INFO

Stack: Ethernet -> IP -> ICMP
------------------------------
MAC Origin: 00:00:00:aa:00:02
MAC Destination: 00:00:00:aa:00:00
IP Origin: 10.0.1.20
IP Destination: 10.0.0.20
TTL: 63
Not Fragmented
Protocol: ICMP (Echo Reply)
------------------------------
Total Size: 74 bytes
------------------------------------------------------
```

4.1.2.1.3 | Fragmentação de ICMPv4

Como já tinha sido referido antes, o *sniffer* permite observar pacotes fragmentados identificando se é o primeiro, intermédio e último fragmento do pacote fragmentado. Esta funcionalidade pode ser testado gerando pacotes cujo tamanho seja superior à **MTU**(*Maximum Transfer Unit*) da interface em questão, geralmente 1500 bytes. Deste modo, a capacidade de identificar fragmentos, através de **MF e/ou Fragment Offset** pode ser testada com um `ping` no qual se usa a flag `-s 2000` para especificar que se pretende gerar pacotes de 2000 bytes de payload.

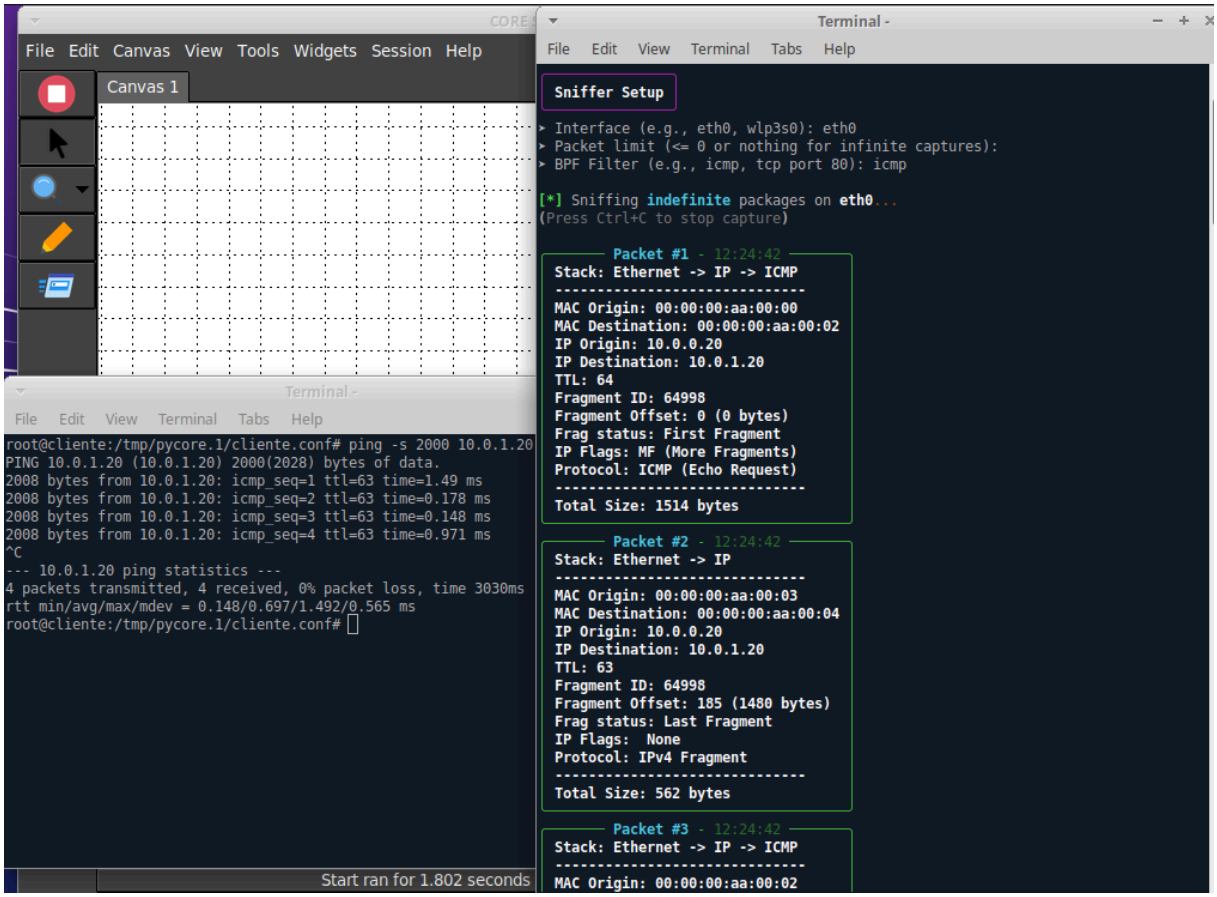


Figura 7 : Demonstração de fragmentação IPv4. O comando `ping -s 2000` gera um pacote superior à **MTU** de 1500 bytes, forçando a divisão em dois fragmentos (Packet 1 e Packet 2).

Como se reconstrói o pacote (Explicação baseada na imagem)

Através dos dados capturados pelo *sniffer* na imagem, a reconstrução do pacote original de 2000 bytes de *payload* baseia-se em três pilares:

1. **Identificação do Fluxo (Fragment ID)**:
   - Ambos os pacotes (1 e 2) possuem o **Fragment ID: 64998**. Isto indica ao recetor que estes pedaços pertencem exatamente à mesma mensagem original.
2. **Ordenação (Fragment Offset)**:
   - No Packet 1, o **Fragment Offset é 0**. Isto indica que este é o início dos dados. O *sniffer* identifica-o como **First Fragment** e mostra que a *flag* **MF (More Fragments) está ativa**, sinalizando que há mais dados a caminho.
   - No Packet 2, o **Fragment Offset é 185** (que corresponde a 1480 bytes de dados deslocados - $185 \times 8 = 1480$). Isto diz ao sistema que este bloco de dados deve ser colado imediatamente após os 1480 bytes do primeiro pacote.
3. **Finalização e Cálculo de Tamanho**:

- O Packet 2 apresenta a flag **MF: None (ou flag desativada)**, o que permite ao *sniffer* classificá-lo como **Last Fragment**.
- Para saber o tamanho total do *payload* original, somamos o ponto de partida do último fragmento (1480 bytes) com o tamanho do seu próprio conteúdo. Na imagem, o Packet 2 tem um tamanho total de 562 bytes (20 bytes de cabeçalho IP + 14 bytes de cabeçalho de **Ethernet** + 528 de dados).
- **Cálculo**: 1480 (*offset*) + 528 (dados do último fragmento) = 2008 bytes.
- **Nota:** O valor final de 2008 bytes inclui os 2000 bytes de payload solicitados no comando `ping -s 2000` mais os 8 bytes do cabeçalho ICMP (que, sendo o protocolo de transporte neste caso, só é incluído no primeiro fragmento). Assim: $2008 - 8 = 2000$ bytes de dados puros.

4.1.2.2 | Camada de Transporte e Aplicação (L4/L7)

Pode-se agora testar a implementação de alguns protocolos de camadas superiores, permitindo analisar a forma como o sniffer trata o as flags do **TCP** e o conteúdo deste e do **UDP**, quando não encriptado.

4.1.2.2.1 | TCP (Transmission Control Protocol)

Para validar o suporte ao protocolo **TCP**, utilizou-se o utilitário netcat (nc). No **Servidor**, iniciou-se um serviço de escuta na porta 50000 através do comando `nc -l -p 50000`. O **Cliente** estabeleceu a ligação com `nc 10.0.1.20 50000`, permitindo a transmissão de mensagens bidirecionais.

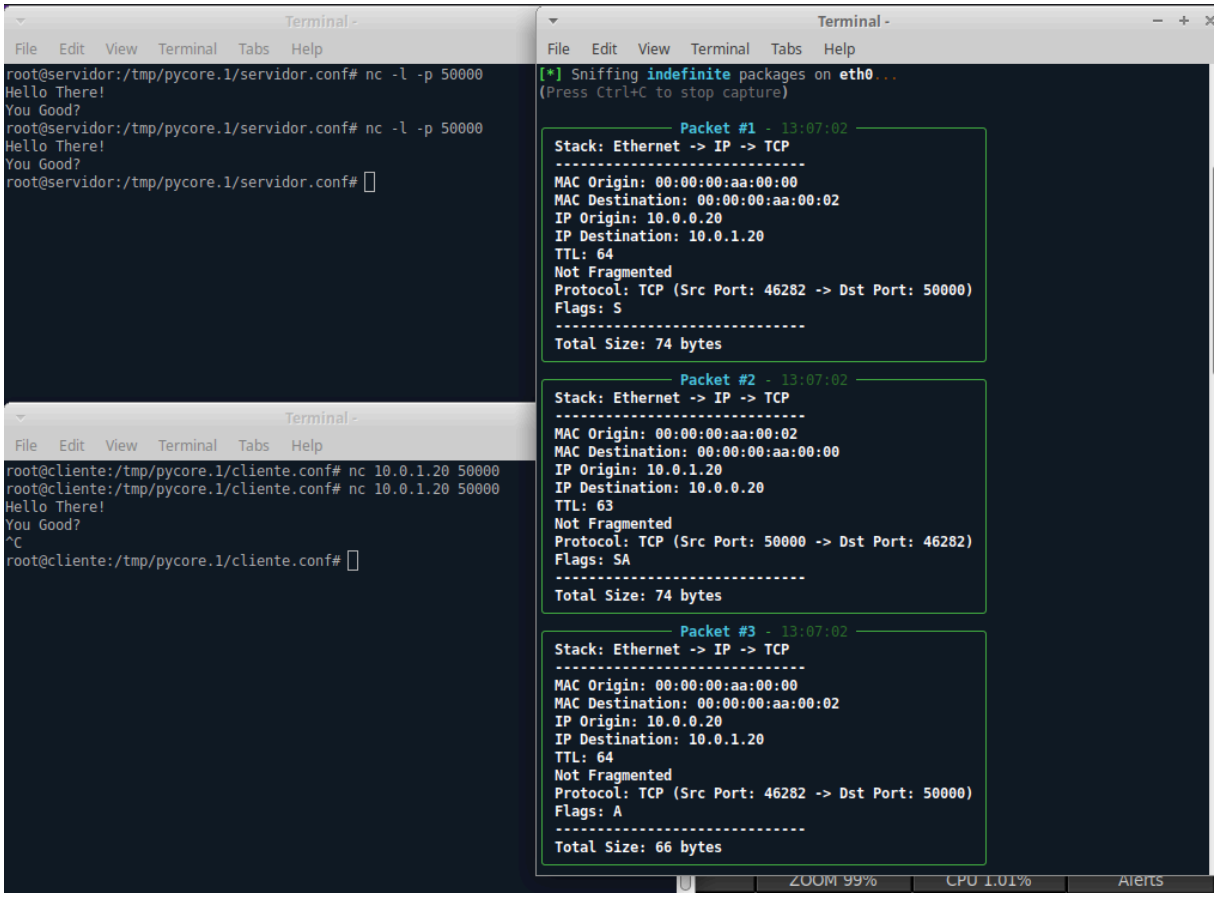


Figura 8 : Fase de estabelecimento de ligação (Handshake) do TCP

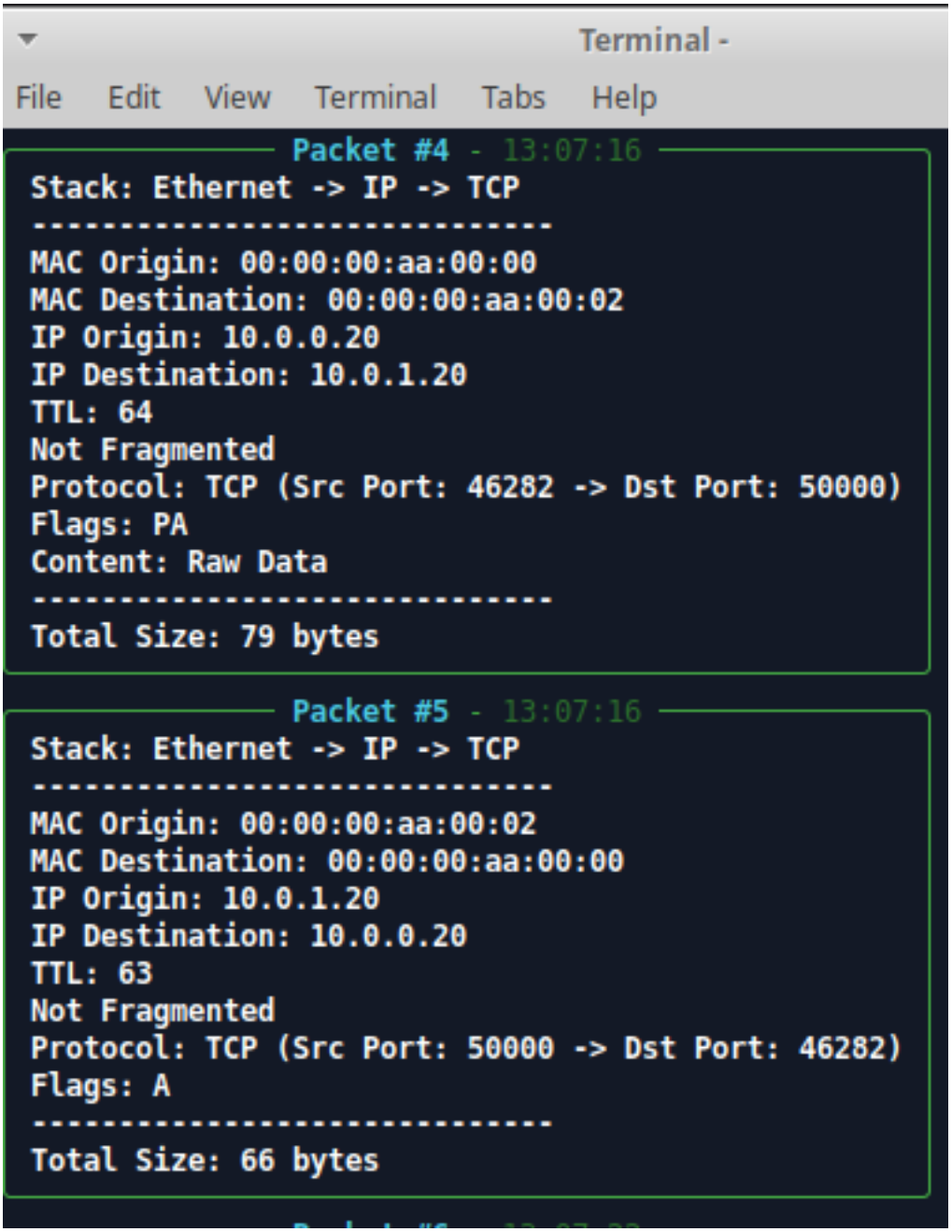


Figura 9 : Fase de transmissão de dados e confirmação

Na Figura 8, o *sniffer* exibe claramente as *flags* durante o **Three-Way Handshake**. Analisando os pacotes 1 a 3, observa-se a sequência exata de estabelecimento de ligação: **S (SYN) -> SA (SYN-ACK) -> A (ACK)** já referida na Figura 3.

Na Figura 9, observa-se a fase de transferência de informação. O pacote 4 mostra o envio de dados do cliente com as *flags* **PA** (PSH-ACK), onde a *flag Push* indica que os dados devem ser entregues imediatamente à aplicação. O pacote 5 mostra a resposta do servidor apenas com a *flag* **A** (ACK), confirmando a receção bem-sucedida dos dados.

4.1.2.2.2 | HTTP (TCP Porta 80)

Este teste permite determinar não só se a deteção de protocolos de layer 7 com base no protocolo de transporte e portas conhecidas está corretamente implementado como também permite identificar, mais uma vez, o método de estabelecimento da ligação TCP.

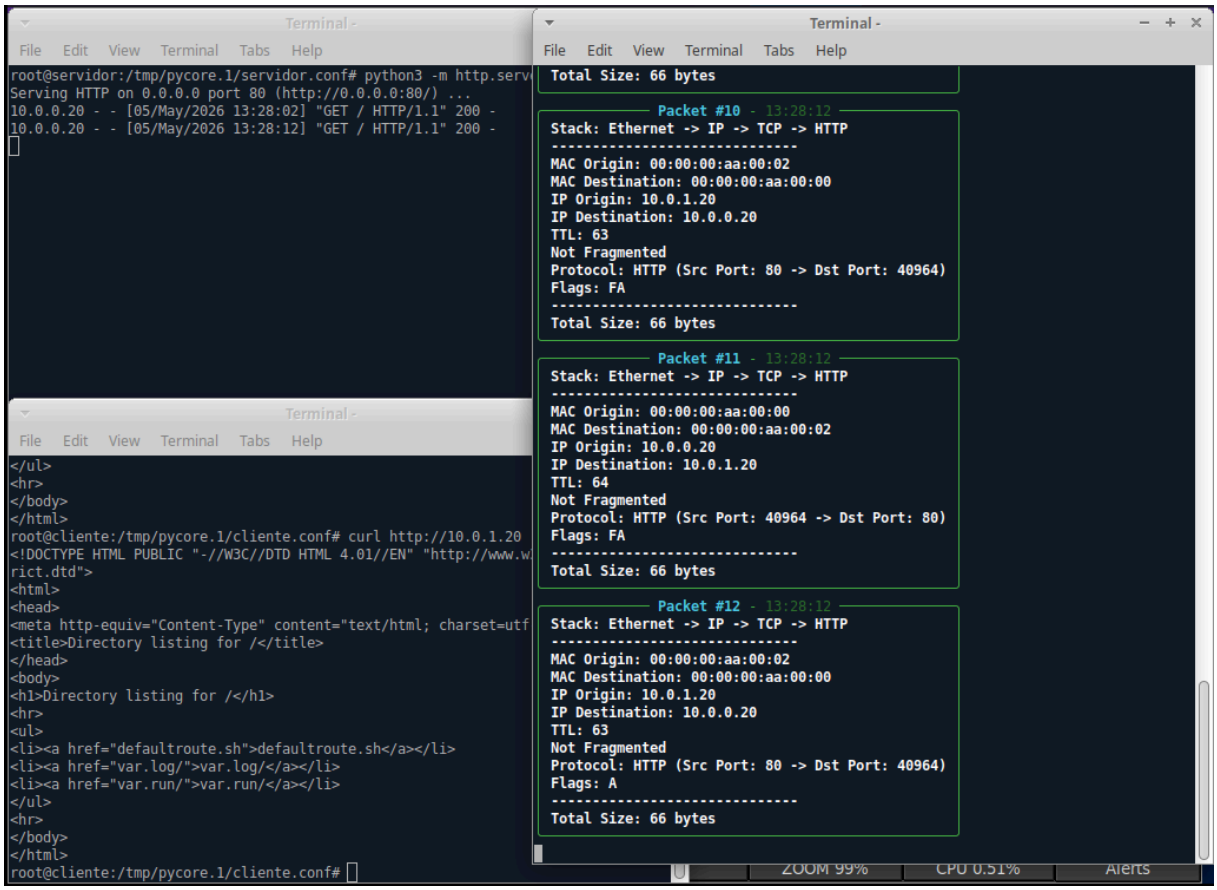


Figura 10 : Teste da implementação de suporte a HTTP.

Na Figura 10, vê-se que o Servidor está, de facto, a correr um servidor HTTP, via o módulo http do python, fazendo o Cliente um pedido via curl . Tal pode ser observado em maior detalhe no excerto do log.txt abaixo:

```
--- Capture Logs ---

==================== SNIFFER NEW SESSION START ====================

------------------------------05/05/2026 13:32:37

--- Capture 1 ---
[1] [05/05/2026 13:32:40]
Ether / IP / TCP 10.0.0.20:57956 > 10.0.1.20:http S
```

```
DETAILED PACKET INFO

Stack: Ethernet -> IP -> TCP -> HTTP
------------------------------
MAC Origin: 00:00:00:aa:00:00
MAC Destination: 00:00:00:aa:00:02
IP Origin: 10.0.0.20
IP Destination: 10.0.1.20
TTL: 64
Not Fragmented
Protocol: HTTP (Src Port: 57956 -> Dst Port: 80)
Flags: S
------------------------------
Total Size: 74 bytes
------------------------------------------------------------
[2] [05/05/2026 13:32:40]
Ether / IP / TCP 10.0.1.20:http > 10.0.0.20:57956 SA

DETAILED PACKET INFO

Stack: Ethernet -> IP -> TCP -> HTTP
------------------------------
MAC Origin: 00:00:00:aa:00:02
MAC Destination: 00:00:00:aa:00:00
IP Origin: 10.0.1.20
IP Destination: 10.0.0.20
TTL: 63
Not Fragmented
Protocol: HTTP (Src Port: 80 -> Dst Port: 57956)
Flags: SA
------------------------------
Total Size: 74 bytes
------------------------------------------------------------
[3] [05/05/2026 13:32:40]
Ether / IP / TCP 10.0.0.20:57956 > 10.0.1.20:http A

DETAILED PACKET INFO

Stack: Ethernet -> IP -> TCP -> HTTP
------------------------------
MAC Origin: 00:00:00:aa:00:00
MAC Destination: 00:00:00:aa:00:02
IP Origin: 10.0.0.20
IP Destination: 10.0.1.20
TTL: 64
Not Fragmented
Protocol: HTTP (Src Port: 57956 -> Dst Port: 80)
Flags: A
```

```
------------------------------
Total Size: 66 bytes
------------------------------------------------------------
[4] [05/05/2026 13:32:40]
Ether / IP / TCP 10.0.0.20:57956 > 10.0.1.20:http PA / Raw

DETAILED PACKET INFO

Stack: Ethernet -> IP -> TCP -> HTTP
------------------------------
MAC Origin: 00:00:00:aa:00:00
MAC Destination: 00:00:00:aa:00:02
IP Origin: 10.0.0.20
IP Destination: 10.0.1.20
TTL: 64
Not Fragmented
Protocol: HTTP (Src Port: 57956 -> Dst Port: 80)
Flags: PA
Content: Raw Data
------------------------------
Total Size: 139 bytes
------------------------------------------------------------
[5] [05/05/2026 13:32:40]
Ether / IP / TCP 10.0.1.20:http > 10.0.0.20:57956 A

DETAILED PACKET INFO

Stack: Ethernet -> IP -> TCP -> HTTP
------------------------------
MAC Origin: 00:00:00:aa:00:02
MAC Destination: 00:00:00:aa:00:00
IP Origin: 10.0.1.20
IP Destination: 10.0.0.20
TTL: 63
Not Fragmented
Protocol: HTTP (Src Port: 80 -> Dst Port: 57956)
Flags: A
------------------------------
Total Size: 66 bytes
------------------------------------------------------------
[6] [05/05/2026 13:32:40]
Ether / IP / TCP 10.0.1.20:http > 10.0.0.20:57956 PA / Raw

DETAILED PACKET INFO

Stack: Ethernet -> IP -> TCP -> HTTP
------------------------------
MAC Origin: 00:00:00:aa:00:02
```

```
MAC Destination: 00:00:00:aa:00:00
IP Origin: 10.0.1.20
IP Destination: 10.0.0.20
TTL: 63
Not Fragmented
Protocol: HTTP (Src Port: 80 -> Dst Port: 57956)
Flags: PA
Content: Raw Data
------------------------------
Total Size: 222 bytes
------------------------------------------------------------
[7] [05/05/2026 13:32:40]
Ether / IP / TCP 10.0.0.20:57956 > 10.0.1.20:http A

DETAILED PACKET INFO

Stack: Ethernet -> IP -> TCP -> HTTP
------------------------------
MAC Origin: 00:00:00:aa:00:00
MAC Destination: 00:00:00:aa:00:02
IP Origin: 10.0.0.20
IP Destination: 10.0.1.20
TTL: 64
Not Fragmented
Protocol: HTTP (Src Port: 57956 -> Dst Port: 80)
Flags: A
------------------------------
Total Size: 66 bytes
------------------------------------------------------------
[8] [05/05/2026 13:32:40]
Ether / IP / TCP 10.0.1.20:http > 10.0.0.20:57956 PA / Raw

DETAILED PACKET INFO

Stack: Ethernet -> IP -> TCP -> HTTP
------------------------------
MAC Origin: 00:00:00:aa:00:02
MAC Destination: 00:00:00:aa:00:00
IP Origin: 10.0.1.20
IP Destination: 10.0.0.20
TTL: 63
Not Fragmented
Protocol: HTTP (Src Port: 80 -> Dst Port: 57956)
Flags: PA
Content: Raw Data
------------------------------
Total Size: 500 bytes
------------------------------------------------------------
```

```
[9] [05/05/2026 13:32:40]
Ether / IP / TCP 10.0.0.20:57956 > 10.0.1.20:http A

DETAILED PACKET INFO

Stack: Ethernet -> IP -> TCP -> HTTP
------------------------------
MAC Origin: 00:00:00:aa:00:00
MAC Destination: 00:00:00:aa:00:02
IP Origin: 10.0.0.20
IP Destination: 10.0.1.20
TTL: 64
Not Fragmented
Protocol: HTTP (Src Port: 57956 -> Dst Port: 80)
Flags: A
------------------------------
Total Size: 66 bytes
------------------------------------------------------------
[10] [05/05/2026 13:32:40]
Ether / IP / TCP 10.0.1.20:http > 10.0.0.20:57956 FA

DETAILED PACKET INFO

Stack: Ethernet -> IP -> TCP -> HTTP
------------------------------
MAC Origin: 00:00:00:aa:00:02
MAC Destination: 00:00:00:aa:00:00
IP Origin: 10.0.1.20
IP Destination: 10.0.0.20
TTL: 63
Not Fragmented
Protocol: HTTP (Src Port: 80 -> Dst Port: 57956)
Flags: FA
------------------------------
Total Size: 66 bytes
------------------------------------------------------------
[11] [05/05/2026 13:32:40]
Ether / IP / TCP 10.0.0.20:57956 > 10.0.1.20:http FA

DETAILED PACKET INFO

Stack: Ethernet -> IP -> TCP -> HTTP
------------------------------
MAC Origin: 00:00:00:aa:00:00
MAC Destination: 00:00:00:aa:00:02
IP Origin: 10.0.0.20
IP Destination: 10.0.1.20
TTL: 64
```

```
Not Fragmented
Protocol: HTTP (Src Port: 57956 -> Dst Port: 80)
Flags: FA
------------------------------
Total Size: 66 bytes
------------------------------------------------------
[12] [05/05/2026 13:32:40]
Ether / IP / TCP 10.0.1.20:http > 10.0.0.20:57956 A

DETAILED PACKET INFO

Stack: Ethernet -> IP -> TCP -> HTTP
------------------------------
MAC Origin: 00:00:00:aa:00:02
MAC Destination: 00:00:00:aa:00:00
IP Origin: 10.0.1.20
IP Destination: 10.0.0.20
TTL: 63
Not Fragmented
Protocol: HTTP (Src Port: 80 -> Dst Port: 57956)
Flags: A
------------------------------
Total Size: 66 bytes
```

Análise Estruturada da Captura:

A análise do ficheiro de registo permite segmentar a comunicação em três fases distintas:

1. **Estabelecimento (Handshake)**:
   - Os pacotes [1] a [3] mostram o **Three-Way Handshake** (*Flags* **S, SA, A**). É interessante notar que, embora o protocolo de transporte seja **TCP**, o *sniffer* já o classifica na *stack* como **HTTP** devido à identificação da porta 80.
2. **Transferência de Dados**:
   - No pacote [4], o cliente envia o pedido (GET /), visível pela *flag* **PA** (Push-Ack) e pela indicação de **Content: Raw Data**.
   - Os pacotes [6] e [8] contêm a resposta do servidor com o conteúdo HTML da página, também identificados com a *flag* **PA** e um tamanho total superior (até 500 bytes), indicando o transporte de informação útil.
3. **Encerramento da Ligação**:
   - Os pacotes [10] a [12] demonstram o fecho ordenado da sessão através das *flags* **FA** (Fin-Ack).

4.1.2.2.3 | DNS (UDP Porta 53)

Para concluir os testes de protocolos com suporte explícito, validou-se a implementação da análise de **DNS** sobre **UDP**. Utilizou-se o utilitário `dig` para gerar *queries* destinadas ao servidor (10.0.1.20). Embora o pedido falhe devido à inexistência de conectividade real à Internet na rede emulada, o tráfego gerado é suficiente para testar a capacidade de inspeção da ferramenta.

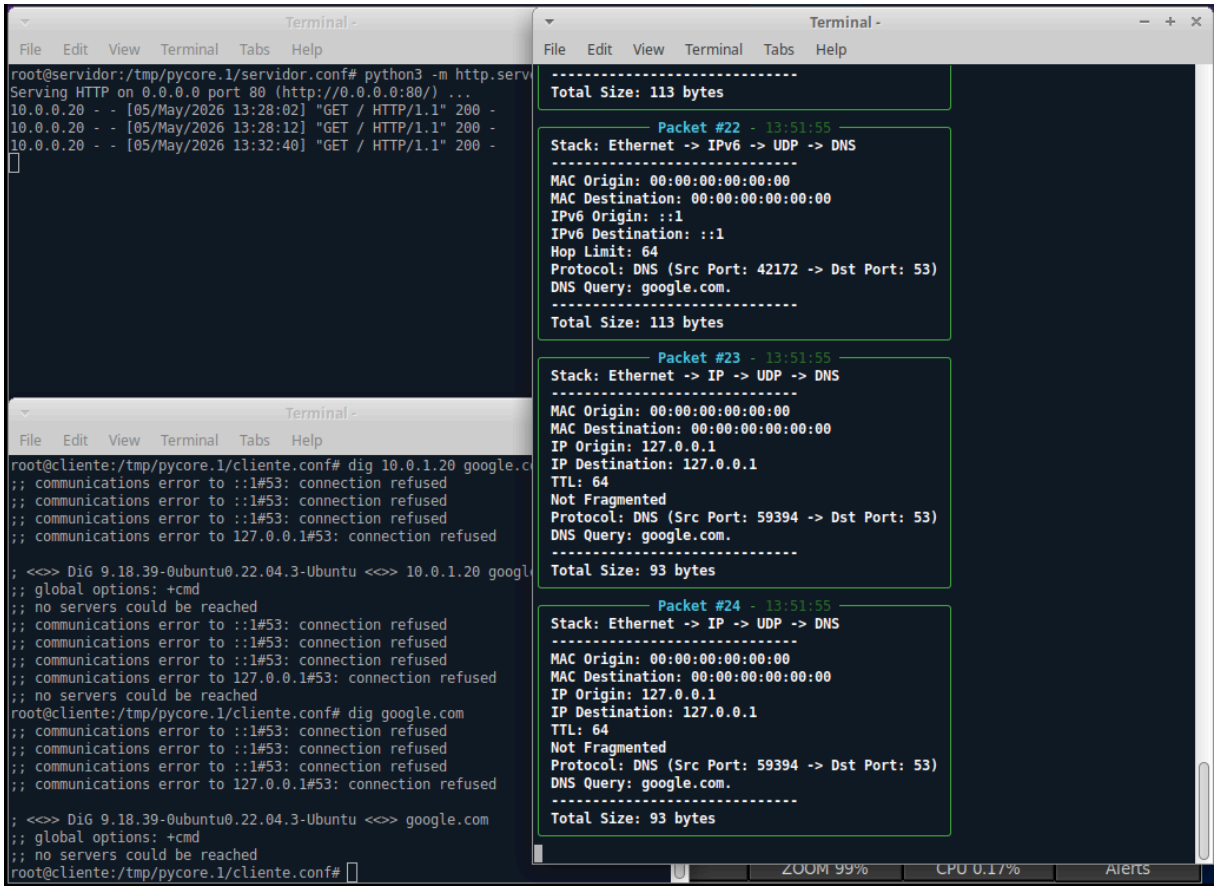


Figura 11 : Captura de pacotes **DNS**. O *sniffer* demonstra versatilidade ao identificar *queries* tanto em tráfego IPv4 (Packet 23) como em IPv6 (Packet 22).

Como se pode observar na figura, o *sniffer* identifica corretamente a porta de destino 53 e extrai o conteúdo da *query* (neste caso, google.com.). É de notar que a ferramenta foi capaz de processar o pedido **DNS** efetuado via **IPv6 Loopback** (::1), extraindo metadados específicos como o **Hop Limit**, o que demonstra a robustez da implementação em diferentes versões do protocolo IP.

Nota extra: Para capturar cada tipo de protocolo mencionado anteriormente, foram utilizados os seguintes filtros (BPF):

- ARP - arp
- ICMP - icmp

- TCP - tcp
- TCP -> HTTP - tcp port 80
- UDP -> DNS - udp port 53

5 | Parte B — Transposição para a interface real do PC

Q: O *sniffer* deve ser executado na interface real do computador do aluno e deverá conseguir identificar e capturar todos os protocolos que já estavam presentes no ambiente de emulação *CORE*.

Algumas restrições a apontar:

- O *sniffer* deve ser usado apenas em redes autorizadas (*CORE* e rede do aluno);
- Não é permitido desenvolver funcionalidades de exploração (ex.: *MITM*, *injection*, *deauth*), nem recolher dados sensíveis;
- O foco é a inspeção passiva e interpretação de protocolos.

5.1 | Análise na Interface Real e Obstáculos Encontrados

5.1.1 | Transição do Ambiente CORE para a Interface Real

A transição do ambiente emulado do CORE para uma interface de rede real (Wi-Fi/Ethernet) constituiu uma etapa significativa no desenvolvimento do projeto, uma vez que introduz variáveis que não existem num cenário controlado. Enquanto no CORE toda a topologia é previsível e o tráfego capturado corresponde exclusivamente às ações deliberadamente desencadeadas pelo utilizador, numa interface real o sniffer é exposto ao tráfego de fundo (*background traffic*) gerado pelo sistema operativo, por aplicações em execução e por outros dispositivos na mesma rede local.

A interface utilizada para os testes reais foi a placa de rede sem fios de um computador de um membro, ligada à rede doméstica Wi-Fi.

5.1.2 | Adaptação do Sniffer à Interface Real

Pela ausência de necessidade de alterações estruturais ao motor de captura, foi empregue na interface real o mesmo código utilizado no CORE. A já existente arquitetura modular, com uma divisão clara da captura (`sniffer.py`), análise (`protocol.py`), configuração (`interface.py`) e orquestração (`main.py`), demonstra suficiente versatilidade na operação tanto em interfaces emuladas quanto em interfaces físicas.

A alteração essencial consiste na própria seleção da interface durante a configuração. No CORE, utiliza-se o nome da interface virtual atribuída ao nó (ex.: `eth0`), enquanto que na interface real indica-se o nome da placa de rede física do sistema (ex.: `wlp3s0` em Linux). Esta transição é gerida pelo módulo `interface.py`, que tem a funcionalidade de listar automaticamente todas as interfaces disponíveis no *host*:

```
def get_config():
    rprint(Panel("[bold white]Sniffer Setup[/bold white]",
           border_style="magenta", expand=False))

    iface = input("➤ Interface (e.g., eth0, wlp3s0): ")
```

```
    if (iface not in scapy.get_if_list()): iface = ""

    limit_input = input("➤ Packet limit (<= 0 or nothing for infinite): ")
    limit = int(limit_input) if limit_input.strip() else 0

    bpf = input("➤ BPF Filter (e.g., icmp, tcp port 80): ")

    return iface, limit, bpf
```

5.1.3 | Protocolos Validados na Interface Real

Todos os protocolos previamente validados no ambiente CORE mostraram-se funcionais na interface real. Adicionalmente, foi possível capturar protocolos que não estão facilmente disponíveis na emulação do CORE:

Tabela 2 : Comparação de protocolos validados no CORE e na interface real

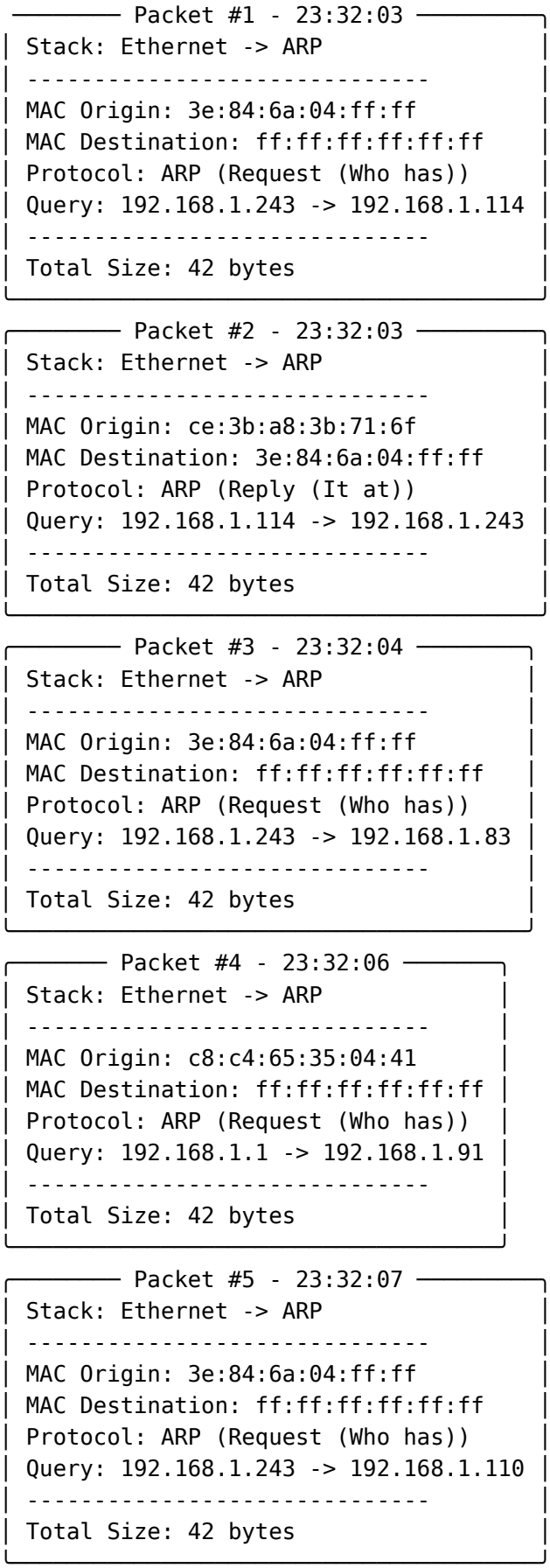
Protocolo	Camada	CORE	Interface Real
ARP	L2/L3	✓	✓
ICMP / ICMPv6	L3	✓	✓
Fragmentação IPv4	L3	✓	✓
TCP (Flags)	L4	✓	✓
HTTP	L7	✓	✓
DNS	L7	✓	✓
DHCP/ DHCPv6	L7	—	✓
QUIC/HTTPS	L7	—	✓

5.1.4 | Testes de Validação na Interface Real

De forma análoga aos testes efetuados no ambiente CORE (secção 4.1.2), foi necessário validar que o sniffer captura e analisa corretamente cada protocolo quando executado na interface real do computador.

5.1.4.1 | ARP

Para validar o **ARP**, limpou-se a cache **ARP**(comando : `sudo ip -s -s neigh flush all`) do sistema e realizou-se um `ping` ao *gateway* da rede doméstica. O sniffer, configurado com o filtro `arp`, capturou corretamente os pacotes de **ARP Request** e **ARP Reply**, demonstrando que a identificação do campo de operação e a extração dos endereços IP envolvidos funcionam de forma idêntica ao ambiente CORE.


Packet #1 - 23:32:03
Stack: Ethernet -> ARP
MAC Origin: 3e:84:6a:04:ff:ff
MAC Destination: ff:ff:ff:ff:ff:ff
Protocol: ARP (Request (Who has))
Query: 192.168.1.243 -> 192.168.1.114
Total Size: 42 bytes
Packet #2 - 23:32:03
Stack: Ethernet -> ARP
MAC Origin: ce:3b:a8:3b:71:6f
MAC Destination: 3e:84:6a:04:ff:ff
Protocol: ARP (Reply (It at))
Query: 192.168.1.114 -> 192.168.1.243
Total Size: 42 bytes
Packet #3 - 23:32:04
Stack: Ethernet -> ARP
MAC Origin: 3e:84:6a:04:ff:ff
MAC Destination: ff:ff:ff:ff:ff:ff
Protocol: ARP (Request (Who has))
Query: 192.168.1.243 -> 192.168.1.83
Total Size: 42 bytes
Packet #4 - 23:32:06
Stack: Ethernet -> ARP
MAC Origin: c8:c4:65:35:04:41
MAC Destination: ff:ff:ff:ff:ff:ff
Protocol: ARP (Request (Who has))
Query: 192.168.1.1 -> 192.168.1.91
Total Size: 42 bytes
Packet #5 - 23:32:07
Stack: Ethernet -> ARP
MAC Origin: 3e:84:6a:04:ff:ff
MAC Destination: ff:ff:ff:ff:ff:ff
Protocol: ARP (Request (Who has))
Query: 192.168.1.243 -> 192.168.1.110
Total Size: 42 bytes

```
┌──────── Packet #6 - 23:32:12 ────────┐
│ Stack: Ethernet -> ARP               │
│ ---------------------------          │
│ MAC Origin: c8:c4:65:35:04:41        │
│ MAC Destination: ff:ff:ff:ff:ff:ff   │
│ Protocol: ARP (Request (Who has))    │
│ Query: 192.168.1.1 -> 192.168.1.91   │
│ ---------------------------          │
│ Total Size: 42 bytes                 │
└──────────────────────────────────────┘
┌──────── Packet #7 - 23:32:13 ────────┐
│ Stack: Ethernet -> ARP               │
│ ---------------------------          │
│ MAC Origin: c8:c4:65:35:04:41        │
│ MAC Destination: ff:ff:ff:ff:ff:ff   │
│ Protocol: ARP (Request (Who has))    │
│ Query: 192.168.1.1 -> 192.168.1.91   │
│ ---------------------------          │
│ Total Size: 42 bytes                 │
└──────────────────────────────────────┘
┌──────── Packet #8 - 23:32:14 ────────┐
│ Stack: Ethernet -> ARP               │
│ ---------------------------          │
│ MAC Origin: c8:c4:65:35:04:41        │
│ MAC Destination: ff:ff:ff:ff:ff:ff   │
│ Protocol: ARP (Request (Who has))    │
│ Query: 192.168.1.1 -> 192.168.1.91   │
│ ---------------------------          │
│ Total Size: 42 bytes                 │
└──────────────────────────────────────┘
┌──────── Packet #9 - 23:32:15 ────────┐
│ Stack: Ethernet -> ARP               │
│ ---------------------------          │
│ MAC Origin: 3e:84:6a:04:ff:ff        │
│ MAC Destination: ff:ff:ff:ff:ff:ff   │
│ Protocol: ARP (Request (Who has))    │
│ Query: 192.168.1.243 -> 192.168.1.114 │
│ ---------------------------          │
│ Total Size: 42 bytes                 │
└──────────────────────────────────────┘
┌──────── Packet #10 - 23:32:15 ───────┐
│ Stack: Ethernet -> ARP               │
│ ---------------------------          │
│ MAC Origin: ce:3b:a8:3b:71:6f        │
│ MAC Destination: 3e:84:6a:04:ff:ff   │
│ Protocol: ARP (Reply (It at))        │
│ Query: 192.168.1.114 -> 192.168.1.243 │
│ ---------------------------          │
│ Total Size: 42 bytes                 │
└──────────────────────────────────────┘
```

Filtro usado: arp

5.1.4.2 | ICMP

Através de um `ping` a um servidor externo (ex.: `8.8.8.8` - **Google public DNS**), validou-se a captura de mensagens **ICMP** na rede real. O *sniffer* identificou corretamente os **Echo Request** e **Echo Reply**, extraindo o campo **TTL** — que, na interface real, reflete a distância efetiva até ao servidor de destino, ao contrário do ambiente CORE onde a topologia é limitada a poucos saltos.

```
┌──────── Packet #1 - 23:33:11 ────────┐
│ Stack: Ethernet -> IP -> ICMP        │
│ ------------------------------       │
│ MAC Origin: ce:3b:a8:3b:71:6f        │
│ MAC Destination: c8:c4:65:35:04:41   │
│ IP Origin: 192.168.1.114             │
│ IP Destination: 8.8.8.8              │
│ TTL: 64                              │
│ Not Fragmented                       │
│ Protocol: ICMP (Echo Request)        │
│ ------------------------------       │
│ Total Size: 98 bytes                 │
└──────────────────────────────────────┘
┌──────── Packet #2 - 23:33:11 ────────┐
│ Stack: Ethernet -> IP -> ICMP        │
│ ------------------------------       │
│ MAC Origin: c8:c4:65:35:04:41        │
│ MAC Destination: ce:3b:a8:3b:71:6f   │
│ IP Origin: 8.8.8.8                   │
│ IP Destination: 192.168.1.114        │
│ TTL: 118                             │
│ Not Fragmented                       │
│ Protocol: ICMP (Echo Reply)          │
│ ------------------------------       │
│ Total Size: 98 bytes                 │
└──────────────────────────────────────┘
┌──────── Packet #3 - 23:33:12 ────────┐
│ Stack: Ethernet -> IP -> ICMP        │
│ ------------------------------       │
│ MAC Origin: ce:3b:a8:3b:71:6f        │
│ MAC Destination: c8:c4:65:35:04:41   │
│ IP Origin: 192.168.1.114             │
│ IP Destination: 8.8.8.8              │
│ TTL: 64                              │
│ Not Fragmented                       │
│ Protocol: ICMP (Echo Request)        │
│ ------------------------------       │
│ Total Size: 98 bytes                 │
└──────────────────────────────────────┘
```

```
┌──────── Packet #4 - 23:33:12 ────────┐
| Stack: Ethernet -> IP -> ICMP       |
| ------------------------------      |
| MAC Origin: c8:c4:65:35:04:41       |
| MAC Destination: ce:3b:a8:3b:71:6f  |
| IP Origin: 8.8.8.8                  |
| IP Destination: 192.168.1.114       |
| TTL: 118                            |
| Not Fragmented                      |
| Protocol: ICMP (Echo Reply)         |
| ------------------------------      |
| Total Size: 98 bytes                |
└─────────────────────────────────────┘
┌──────── Packet #5 - 23:33:16 ────────┐
| Stack: Ethernet -> IP -> ICMP       |
| ------------------------------      |
| MAC Origin: ce:3b:a8:3b:71:6f       |
| MAC Destination: c8:c4:65:35:04:41  |
| IP Origin: 192.168.1.114            |
| IP Destination: 8.8.8.8             |
| TTL: 64                             |
| Not Fragmented                      |
| Protocol: ICMP (Echo Request)       |
| ------------------------------      |
| Total Size: 98 bytes                |
└─────────────────────────────────────┘
┌──────── Packet #6 - 23:33:16 ────────┐
| Stack: Ethernet -> IP -> ICMP       |
| ------------------------------      |
| MAC Origin: c8:c4:65:35:04:41       |
| MAC Destination: ce:3b:a8:3b:71:6f  |
| IP Origin: 8.8.8.8                  |
| IP Destination: 192.168.1.114       |
| TTL: 118                            |
| Not Fragmented                      |
| Protocol: ICMP (Echo Reply)         |
| ------------------------------      |
| Total Size: 98 bytes                |
└─────────────────────────────────────┘
┌──────── Packet #7 - 23:33:17 ────────┐
| Stack: Ethernet -> IP -> ICMP       |
| ------------------------------      |
| MAC Origin: ce:3b:a8:3b:71:6f       |
| MAC Destination: c8:c4:65:35:04:41  |
| IP Origin: 192.168.1.114            |
| IP Destination: 8.8.8.8             |
| TTL: 64                             |
| Not Fragmented                      |
| Protocol: ICMP (Echo Request)       |
| ------------------------------      |
```

```
| Total Size: 98 bytes                |
└─────────────────────────────────────┘

┌──────── Packet #8 - 23:33:17 ────────┐
| Stack: Ethernet -> IP -> ICMP        |
| ------------------------------       |
| MAC Origin: c8:c4:65:35:04:41        |
| MAC Destination: ce:3b:a8:3b:71:6f   |
| IP Origin: 8.8.8.8                   |
| IP Destination: 192.168.1.114        |
| TTL: 118                             |
| Not Fragmented                       |
| Protocol: ICMP (Echo Reply)          |
| ------------------------------       |
| Total Size: 98 bytes                 |
└──────────────────────────────────────┘

┌──────── Packet #9 - 23:33:18 ────────┐
| Stack: Ethernet -> IP -> ICMP        |
| ------------------------------       |
| MAC Origin: ce:3b:a8:3b:71:6f        |
| MAC Destination: c8:c4:65:35:04:41   |
| IP Origin: 192.168.1.114             |
| IP Destination: 8.8.8.8              |
| TTL: 64                              |
| Not Fragmented                       |
| Protocol: ICMP (Echo Request)        |
| ------------------------------       |
| Total Size: 98 bytes                 |
└──────────────────────────────────────┘

┌──────── Packet #10 - 23:33:18 ───────┐
| Stack: Ethernet -> IP -> ICMP        |
| ------------------------------       |
| MAC Origin: c8:c4:65:35:04:41        |
| MAC Destination: ce:3b:a8:3b:71:6f   |
| IP Origin: 8.8.8.8                   |
| IP Destination: 192.168.1.114        |
| TTL: 118                             |
| Not Fragmented                       |
| Protocol: ICMP (Echo Reply)          |
| ------------------------------       |
| Total Size: 98 bytes                 |
└──────────────────────────────────────┘
```

Filtro usado: icmp

5.1.4.3 | DNS

Ao utilizar o utilitário `dig`(novamente) para resolver domínios (ex.: `google.com`), o *sniffer* capturou **queries DNS** reais destinadas ao servidor **DNS** configurado na rede doméstica. À semelhança do teste no CORE, a ferramenta extraiu corretamente o campo `DNS Query` com o nome do domínio consultado. É de realçar que, na interface real, o

volume de tráfego **DNS** é significativamente superior, uma vez que o sistema operativo e as aplicações em segundo plano geram consultas **DNS** periodicamente.

```
┌──────────── Packet #1 - 23:34:18 ────────────┐
│ Stack: Ethernet -> IPv6 -> UDP -> DNS        │
│ ---------------------------                  │
│ MAC Origin: ce:3b:a8:3b:71:6f                │
│ MAC Destination: c8:c4:65:35:04:41           │
│ IPv6 Origin: fe80::806:a25a:e8f2:57f5        │
│ IPv6 Destination: fe80::1                    │
│ Hop Limit: 64                                │
│ Protocol: DNS (Src Port: 57283 -> Dst Port: 53) │
│ DNS Query: google.com.                       │
│ ---------------------------                  │
│ Total Size: 101 bytes                        │
└──────────────────────────────────────────────┘
┌──────────── Packet #2 - 23:34:18 ────────────┐
│ Stack: Ethernet -> IPv6 -> UDP -> DNS        │
│ ---------------------------                  │
│ MAC Origin: c8:c4:65:35:04:41                │
│ MAC Destination: ce:3b:a8:3b:71:6f           │
│ IPv6 Origin: fe80::1                         │
│ IPv6 Destination: fe80::806:a25a:e8f2:57f5   │
│ Hop Limit: 64                                │
│ Protocol: DNS (Src Port: 53 -> Dst Port: 57283) │
│ DNS Query: google.com.                       │
│ ---------------------------                  │
│ Total Size: 117 bytes                        │
└──────────────────────────────────────────────┘
┌──────────── Packet #3 - 23:34:20 ────────────┐
│ Stack: Ethernet -> IPv6 -> UDP -> DNS        │
│ ---------------------------                  │
│ MAC Origin: ce:3b:a8:3b:71:6f                │
│ MAC Destination: c8:c4:65:35:04:41           │
│ IPv6 Origin: fe80::806:a25a:e8f2:57f5        │
│ IPv6 Destination: fe80::1                    │
│ Hop Limit: 64                                │
│ Protocol: DNS (Src Port: 64618 -> Dst Port: 53) │
│ DNS Query: google.com.                       │
│ ---------------------------                  │
│ Total Size: 101 bytes                        │
└──────────────────────────────────────────────┘
┌──────────── Packet #4 - 23:34:20 ────────────┐
│ Stack: Ethernet -> IPv6 -> UDP -> DNS        │
│ ---------------------------                  │
│ MAC Origin: c8:c4:65:35:04:41                │
│ MAC Destination: ce:3b:a8:3b:71:6f           │
│ IPv6 Origin: fe80::1                         │
│ IPv6 Destination: fe80::806:a25a:e8f2:57f5   │
│ Hop Limit: 64                                │
```

```
| Protocol: DNS (Src Port: 53 -> Dst Port: 64618) |
| DNS Query: google.com.                          |
| ------------------------------                  |
| Total Size: 117 bytes                           |
└─────────────────────────────────────────────────┘

┌──────────────── Packet #5 - 23:34:22 ────────────┐
| Stack: Ethernet -> IPv6 -> UDP -> DNS           |
| ------------------------------                  |
| MAC Origin: ce:3b:a8:3b:71:6f                   |
| MAC Destination: c8:c4:65:35:04:41              |
| IPv6 Origin: fe80::806:a25a:e8f2:57f5           |
| IPv6 Destination: fe80::1                       |
| Hop Limit: 64                                   |
| Protocol: DNS (Src Port: 64703 -> Dst Port: 53) |
| DNS Query: google.com.                          |
| ------------------------------                  |
| Total Size: 101 bytes                           |
└─────────────────────────────────────────────────┘

┌──────────────── Packet #6 - 23:34:22 ────────────┐
| Stack: Ethernet -> IPv6 -> UDP -> DNS           |
| ------------------------------                  |
| MAC Origin: c8:c4:65:35:04:41                   |
| MAC Destination: ce:3b:a8:3b:71:6f              |
| IPv6 Origin: fe80::1                            |
| IPv6 Destination: fe80::806:a25a:e8f2:57f5      |
| Hop Limit: 64                                   |
| Protocol: DNS (Src Port: 53 -> Dst Port: 64703) |
| DNS Query: google.com.                          |
| ------------------------------                  |
| Total Size: 117 bytes                           |
└─────────────────────────────────────────────────┘

┌──────────────── Packet #7 - 23:34:23 ────────────┐
| Stack: Ethernet -> IPv6 -> UDP -> DNS           |
| ------------------------------                  |
| MAC Origin: ce:3b:a8:3b:71:6f                   |
| MAC Destination: c8:c4:65:35:04:41              |
| IPv6 Origin: fe80::806:a25a:e8f2:57f5           |
| IPv6 Destination: fe80::1                       |
| Hop Limit: 64                                   |
| Protocol: DNS (Src Port: 59216 -> Dst Port: 53) |
| DNS Query: google.com.                          |
| ------------------------------                  |
| Total Size: 101 bytes                           |
└─────────────────────────────────────────────────┘

┌──────────────── Packet #8 - 23:34:23 ────────────┐
| Stack: Ethernet -> IPv6 -> UDP -> DNS           |
| ------------------------------                  |
| MAC Origin: c8:c4:65:35:04:41                   |
| MAC Destination: ce:3b:a8:3b:71:6f              |
| IPv6 Origin: fe80::1                            |
```

```
| IPv6 Destination: fe80::806:a25a:e8f2:57f5      |
| Hop Limit: 64                                   |
| Protocol: DNS (Src Port: 53 -> Dst Port: 59216) |
| DNS Query: google.com.                          |
| ------------------------------                  |
| Total Size: 117 bytes                           |
└─────────────────────────────────────────────────┘
┌──────────── Packet #9 - 23:34:24 ────────────┐
| Stack: Ethernet -> IPv6 -> UDP -> DNS        |
| ------------------------------               |
| MAC Origin: ce:3b:a8:3b:71:6f                |
| MAC Destination: c8:c4:65:35:04:41           |
| IPv6 Origin: fe80::806:a25a:e8f2:57f5        |
| IPv6 Destination: fe80::1                    |
| Hop Limit: 64                                |
| Protocol: DNS (Src Port: 54510 -> Dst Port: 53) |
| DNS Query: google.com.                       |
| ------------------------------               |
| Total Size: 101 bytes                        |
└──────────────────────────────────────────────┘
┌──────────── Packet #10 - 23:34:24 ────────────┐
| Stack: Ethernet -> IPv6 -> UDP -> DNS         |
| ------------------------------                |
| MAC Origin: c8:c4:65:35:04:41                 |
| MAC Destination: ce:3b:a8:3b:71:6f            |
| IPv6 Origin: fe80::1                          |
| IPv6 Destination: fe80::806:a25a:e8f2:57f5    |
| Hop Limit: 64                                 |
| Protocol: DNS (Src Port: 53 -> Dst Port: 54510) |
| DNS Query: google.com.                        |
| ------------------------------                |
| Total Size: 117 bytes                         |
└───────────────────────────────────────────────┘
```

Filtro usado: udp port 53

5.1.4.4 | DHCP (Dynamic Host Configuration Protocol)

Na interface real, foi possível observar o processo **DHCP**, que não é reproduzível de forma direta no CORE. Ao renovar o *lease* **DHCP** da interface, o *sniffer* capturou a sequência característica de mensagens **DORA** (**Discover** → **Offer** → **Request** → **ACK**), demonstrando a capacidade da ferramenta em identificar e classificar cada etapa do protocolo:

- **DHCP Discover** — O cliente envia um *broadcast* para localizar servidores **DHCP** disponíveis na rede.
- **DHCP Offer** — O servidor responde com uma proposta de configuração (endereço IP, máscara, *gateway*, **DNS**).

- **DHCP Request** — O cliente aceita a proposta e solicita formalmente a configuração oferecida.
- **DHCP ACK** — O servidor confirma a atribuição, concluindo a negociação.

Nota de Observação: Durante as capturas, foi identificado um pacote com o campo `DHCP Message: 7`. Este valor corresponde a um **DHCP Release**, enviado pelo cliente para libertar o seu endereço IP. Embora o foco da implementação do *sniffer* tenha sido o ciclo de atribuição (**DORA**), a ferramenta demonstrou capacidade de extrair corretamente o código da operação a partir das **DHCP Options**, validando a precisão da dissecação do protocolo.

A implementação da análise **DHCP** no `protocol.py` extrai o tipo de mensagem diretamente das opções do pacote, recorrendo ao dicionário `dhcp_msgs`. A identificação é feita primeiro ao nível do `UDP_MAP` — onde as portas 67 e 68 são mapeadas para `"DHCP"` — e depois refinada através da inspeção da camada `scapy.DHCP`:

```
# -- DHCP (camada 7 -> Aplicação) ---
elif pkt.haslayer(scapy.DHCP):
    options = pkt[scapy.DHCP].options
    msg_type = next((opt[1] for opt in options
                     if isinstance(opt, tuple) and opt[0] == 'message-type'),
                    "Unknown")
    dhcp_msgs = {1: "Discover", 2: "Offer", 3: "Request", 5: "ACK"}
    details.append(f"DHCP Message: {dhcp_msgs.get(msg_type, msg_type)}")
```

```
┌──────────────────── Packet #1 - 23:35:29 ────────────────────┐
| Stack: Ethernet -> IP -> UDP -> BOOTP -> DHCP options -> DHCP |
| ----------------------------                                  |
| MAC Origin: ce:3b:a8:3b:71:6f                                 |
| MAC Destination: c8:c4:65:35:04:41                            |
| IP Origin: 192.168.1.114                                      |
| IP Destination: 192.168.1.1                                   |
| TTL: 64                                                       |
| Not Fragmented                                                |
| Protocol: DHCP (Src Port: 68 -> Dst Port: 67)                 |
| DHCP Message: 7                                               |
| ----------------------------                                  |
| Total Size: 342 bytes                                         |
└──────────────────────────────────────────────────────────────┘
┌──────────────────── Packet #2 - 23:35:29 ────────────────────┐
| Stack: Ethernet -> IP -> UDP -> BOOTP -> DHCP options -> DHCP |
| ----------------------------                                  |
| MAC Origin: ce:3b:a8:3b:71:6f                                 |
| MAC Destination: ff:ff:ff:ff:ff:ff                            |
| IP Origin: 0.0.0.0                                            |
| IP Destination: 255.255.255.255                               |
| TTL: 255                                                      |
| Not Fragmented                                                |
```

```
| Protocol: DHCP (Src Port: 68 -> Dst Port: 67)                |
| DHCP Message: Discover                                       |
| -----------------------------                                |
| Total Size: 342 bytes                                        |
└──────────────────────────────────────────────────────────────┘

┌──────────────────────── Packet #3 - 23:35:30 ────────────────┐
| Stack: Ethernet -> IP -> UDP -> BOOTP -> DHCP options -> DHCP |
| -----------------------------                                |
| MAC Origin: ce:3b:a8:3b:71:6f                                |
| MAC Destination: ff:ff:ff:ff:ff:ff                           |
| IP Origin: 0.0.0.0                                           |
| IP Destination: 255.255.255.255                              |
| TTL: 255                                                     |
| Not Fragmented                                               |
| Protocol: DHCP (Src Port: 68 -> Dst Port: 67)                |
| DHCP Message: Discover                                       |
| -----------------------------                                |
| Total Size: 342 bytes                                        |
└──────────────────────────────────────────────────────────────┘

┌──────────────────────── Packet #4 - 23:35:31 ────────────────┐
| Stack: Ethernet -> IP -> UDP -> BOOTP -> DHCP options -> DHCP |
| -----------------------------                                |
| MAC Origin: c8:c4:65:35:04:41                                |
| MAC Destination: ce:3b:a8:3b:71:6f                           |
| IP Origin: 192.168.1.1                                       |
| IP Destination: 192.168.1.114                                |
| TTL: 64                                                      |
| Not Fragmented                                               |
| Protocol: DHCP (Src Port: 67 -> Dst Port: 68)                |
| DHCP Message: Offer                                          |
| -----------------------------                                |
| Total Size: 590 bytes                                        |
└──────────────────────────────────────────────────────────────┘

┌──────────────────────── Packet #5 - 23:35:32 ────────────────┐
| Stack: Ethernet -> IP -> UDP -> BOOTP -> DHCP options -> DHCP |
| -----------------------------                                |
| MAC Origin: ce:3b:a8:3b:71:6f                                |
| MAC Destination: ff:ff:ff:ff:ff:ff                           |
| IP Origin: 0.0.0.0                                           |
| IP Destination: 255.255.255.255                              |
| TTL: 255                                                     |
| Not Fragmented                                               |
| Protocol: DHCP (Src Port: 68 -> Dst Port: 67)                |
| DHCP Message: Request                                        |
| -----------------------------                                |
| Total Size: 342 bytes                                        |
└──────────────────────────────────────────────────────────────┘

┌──────────────────────── Packet #6 - 23:35:32 ────────────────┐
| Stack: Ethernet -> IP -> UDP -> BOOTP -> DHCP options -> DHCP |
| -----------------------------                                |
```

```
| MAC Origin: c8:c4:65:35:04:41                                    |
| MAC Destination: ce:3b:a8:3b:71:6f                               |
| IP Origin: 192.168.1.1                                           |
| IP Destination: 192.168.1.114                                    |
| TTL: 64                                                          |
| Not Fragmented                                                   |
| Protocol: DHCP (Src Port: 67 -> Dst Port: 68)                    |
| DHCP Message: ACK                                                |
| ------------------------------                                   |
| Total Size: 590 bytes                                            |
└──────────────────────────────────────────────────────────────────┘

┌──────────────────────── Packet #7 - 23:35:36 ────────────────────────┐
| Stack: Ethernet -> IP -> UDP -> BOOTP -> DHCP options -> DHCP        |
| ------------------------------                                       |
| MAC Origin: ce:3b:a8:3b:71:6f                                        |
| MAC Destination: c8:c4:65:35:04:41                                   |
| IP Origin: 192.168.1.114                                             |
| IP Destination: 192.168.1.1                                          |
| TTL: 64                                                              |
| Not Fragmented                                                       |
| Protocol: DHCP (Src Port: 68 -> Dst Port: 67)                        |
| DHCP Message: 7                                                      |
| ------------------------------                                       |
| Total Size: 342 bytes                                                |
└──────────────────────────────────────────────────────────────────────┘

┌──────────────────────── Packet #8 - 23:35:36 ────────────────────────┐
| Stack: Ethernet -> IP -> UDP -> BOOTP -> DHCP options -> DHCP        |
| ------------------------------                                       |
| MAC Origin: ce:3b:a8:3b:71:6f                                        |
| MAC Destination: ff:ff:ff:ff:ff:ff                                   |
| IP Origin: 0.0.0.0                                                   |
| IP Destination: 255.255.255.255                                      |
| TTL: 255                                                             |
| Not Fragmented                                                       |
| Protocol: DHCP (Src Port: 68 -> Dst Port: 67)                        |
| DHCP Message: Discover                                               |
| ------------------------------                                       |
| Total Size: 342 bytes                                                |
└──────────────────────────────────────────────────────────────────────┘

┌──────────────────────── Packet #9 - 23:35:37 ────────────────────────┐
| Stack: Ethernet -> IP -> UDP -> BOOTP -> DHCP options -> DHCP        |
| ------------------------------                                       |
| MAC Origin: ce:3b:a8:3b:71:6f                                        |
| MAC Destination: ff:ff:ff:ff:ff:ff                                   |
| IP Origin: 0.0.0.0                                                   |
| IP Destination: 255.255.255.255                                      |
| TTL: 255                                                             |
| Not Fragmented                                                       |
| Protocol: DHCP (Src Port: 68 -> Dst Port: 67)                        |
| DHCP Message: Discover                                               |
```

```
| ----------------------------                                          |
| Total Size: 342 bytes                                                 |
└───────────────────────────────────────────────────────────────────────┘

┌──────────────────── Packet #10 - 23:35:38 ────────────────────┐
| Stack: Ethernet -> IP -> UDP -> BOOTP -> DHCP options -> DHCP |
| ----------------------------                                  |
| MAC Origin: c8:c4:65:35:04:41                                 |
| MAC Destination: ce:3b:a8:3b:71:6f                            |
| IP Origin: 192.168.1.1                                        |
| IP Destination: 192.168.1.114                                 |
| TTL: 64                                                       |
| Not Fragmented                                                |
| Protocol: DHCP (Src Port: 67 -> Dst Port: 68)                 |
| DHCP Message: Offer                                           |
| ----------------------------                                  |
| Total Size: 590 bytes                                         |
└───────────────────────────────────────────────────────────────┘
```

Filtro usado: udp port 67 && udp port 68

5.1.4.5 | QUIC/HTTPS

Na interface real, ao navegar normalmente na Internet, verificou-se uma presença significativa de tráfego na porta UDP 443, correspondente ao protocolo **QUIC**. O *sniffer* identifica corretamente este tráfego através do `UDP_MAP`, classificando-o como `QUIC/HTTPS`. Esta observação é particularmente relevante, pois confirma a tendência atual da migração de serviços Web (como o *Google* e o *YouTube*) do tradicional **HTTPS** sobre **TCP** para o **QUIC** sobre **UDP**.

A distinção entre **HTTPS** convencional e **QUIC** é possível graças à separação de mapeamentos implementada no `protocol.py`:

```
TCP_MAP = {80: "HTTP", 443: "HTTPS", 22: "SSH", 21: "FTP",
           23: "Telnet", 25: "SMTP", 110: "POP3"}
UDP_MAP = {53: "DNS", 67: "DHCP", 68: "DHCP", 546: "DHCPv6",
           547: "DHCPv6", 443: "QUIC/HTTPS", 123: "NTP", 1900: "SSDP"}
```

Assim, quando o tráfego na porta 443 transita sobre **TCP**, é classificado como `HTTPS`; quando transita sobre **UDP**, é identificado como `QUIC/HTTPS`.

```
┌──────────────────── Packet #25 - 09:55:47 ────────────────────┐
| Stack: Ethernet -> IPv6 -> UDP -> QUIC/HTTPS                  |
| ----------------------------                                  |
| MAC Origin: 74:97:79:79:10:99                                 |
| MAC Destination: 04:b0:e7:53:d2:6b                            |
| IPv6 Origin: 2001:818:eadc:e700:f4d4:2ad5:e4b2:21             |
| IPv6 Destination: 2a00:1450:4004::a                           |
| Hop Limit: 64                                                 |
| Protocol: QUIC/HTTPS (Src Port: 33654 -> Dst Port: 443)       |
```

```
| ----------------------------                                    |
| Total Size: 102 bytes                                           |
└─────────────────────────────────────────────────────────────────┘

┌──────────────────── Packet #26 - 09:55:47 ──────────────────────┐
| Stack: Ethernet -> IPv6 -> UDP -> QUIC/HTTPS                    |
| ----------------------------                                    |
| MAC Origin: 04:b0:e7:53:d2:6b                                   |
| MAC Destination: 74:97:79:79:10:99                              |
| IPv6 Origin: 2a00:1450:4004::a                                  |
| IPv6 Destination: 2001:818:eadc:e700:f4d4:2ad5:e4b2:21          |
| Hop Limit: 60                                                   |
| Protocol: QUIC/HTTPS (Src Port: 443 -> Dst Port: 33654)         |
| ----------------------------                                    |
| Total Size: 283 bytes                                           |
└─────────────────────────────────────────────────────────────────┘

┌──────────────────── Packet #27 - 09:55:47 ──────────────────────┐
| Stack: Ethernet -> IPv6 -> UDP -> QUIC/HTTPS                    |
| ----------------------------                                    |
| MAC Origin: 04:b0:e7:53:d2:6b                                   |
| MAC Destination: 74:97:79:79:10:99                              |
| IPv6 Origin: 2a00:1450:4004::a                                  |
| IPv6 Destination: 2001:818:eadc:e700:f4d4:2ad5:e4b2:21          |
| Hop Limit: 60                                                   |
| Protocol: QUIC/HTTPS (Src Port: 443 -> Dst Port: 33654)         |
| ----------------------------                                    |
| Total Size: 86 bytes                                            |
└─────────────────────────────────────────────────────────────────┘

┌──────────────────── Packet #28 - 09:55:47 ──────────────────────┐
| Stack: Ethernet -> IPv6 -> UDP -> QUIC/HTTPS                    |
| ----------------------------                                    |
| MAC Origin: 74:97:79:79:10:99                                   |
| MAC Destination: 04:b0:e7:53:d2:6b                              |
| IPv6 Origin: 2001:818:eadc:e700:f4d4:2ad5:e4b2:21               |
| IPv6 Destination: 2a00:1450:4004::a                             |
| Hop Limit: 64                                                   |
| Protocol: QUIC/HTTPS (Src Port: 33654 -> Dst Port: 443)         |
| ----------------------------                                    |
| Total Size: 95 bytes                                            |
└─────────────────────────────────────────────────────────────────┘

┌──────────────────── Packet #29 - 09:55:47 ──────────────────────┐
| Stack: Ethernet -> IPv6 -> UDP -> QUIC/HTTPS                    |
| ----------------------------                                    |
| MAC Origin: 04:b0:e7:53:d2:6b                                   |
| MAC Destination: 74:97:79:79:10:99                              |
| IPv6 Origin: 2a00:1450:4004::a                                  |
| IPv6 Destination: 2001:818:eadc:e700:f4d4:2ad5:e4b2:21          |
| Hop Limit: 60                                                   |
| Protocol: QUIC/HTTPS (Src Port: 443 -> Dst Port: 33654)         |
| ----------------------------                                    |
| Total Size: 88 bytes                                            |
```

```
└──────────────────────────────────────────────────────────┘
┌──────────────── Packet #30 - 09:55:49 ────────────────┐
│ Stack: Ethernet -> IPv6 -> TCP -> HTTPS               │
│ ----------------------------                          │
│ MAC Origin: 74:97:79:79:10:99                         │
│ MAC Destination: 04:b0:e7:53:d2:6b                    │
│ IPv6 Origin: 2001:818:eadc:e700:f4d4:2ad5:e4b2:21     │
│ IPv6 Destination: 2a03:2880:f252:c8:face:b00c:0:167   │
│ Hop Limit: 64                                         │
│ Protocol: HTTPS (Src Port: 35782 -> Dst Port: 443)    │
│ Flags: PA                                             │
│ Content: Raw Data                                     │
│ ----------------------------                          │
│ Total Size: 156 bytes                                 │
└───────────────────────────────────────────────────────┘
┌──────────────── Packet #31 - 09:55:49 ────────────────┐
│ Stack: Ethernet -> IPv6 -> TCP -> HTTPS               │
│ ----------------------------                          │
│ MAC Origin: 04:b0:e7:53:d2:6b                         │
│ MAC Destination: 74:97:79:79:10:99                    │
│ IPv6 Origin: 2a03:2880:f252:c8:face:b00c:0:167        │
│ IPv6 Destination: 2001:818:eadc:e700:f4d4:2ad5:e4b2:21 │
│ Hop Limit: 57                                         │
│ Protocol: HTTPS (Src Port: 443 -> Dst Port: 35782)    │
│ Flags: A                                              │
│ ----------------------------                          │
│ Total Size: 86 bytes                                  │
└───────────────────────────────────────────────────────┘
```

Filtro Utilizado: port 443 (Para capturar ambos **HTTPS** em **TCP** e **QUIC/HTTPS** em **UDP**)

5.1.5 | Diferenças Observadas em Relação ao Ambiente CORE

A execução na interface real revelou diferenças substanciais em relação ao ambiente emulado:

1. **Volume e Diversidade de Tráfego:** No CORE, cada pacote capturado era o resultado direto de uma ação deliberada (ex.: `ping`, `curl`). Na interface real, o sniffer é exposto ao tráfego de fundo permanente — consultas **DNS** periódicas do sistema operativo, *keep-alives* **TCP** de aplicações em segundo plano, pacotes **SSDP** de dispositivos *IoT* na rede, e tráfego **IPv6** de *neighbor discovery*. Esta situação tornou a utilização de filtros **BPF** absolutamente essencial para isolar o tráfego de interesse.
2. **Encapsulamento Wi-Fi:** No CORE, todos os pacotes são encapsulados em frames Ethernet II convencionais. Numa interface **Wi-Fi** em modo gerido (*Managed Mode* — o modo padrão), o sistema operativo e os controladores da placa de rede abstraem o tráfego rádio, encapsulando-o em cabeçalhos Ethernet II fictícios (processo conhecido como *Ethernet Emulation*). Consequentemente, o *sniffer* observa pacotes aparente-

mente idênticos aos do CORE, sem acesso aos cabeçalhos nativos do protocolo **IEEE 802.11** tal como foi referido na Nota sobre Wifi(IEEE 802.11) 1.

3. **Pacotes "Mascarados":** Conforme documentado na secção 3.2 do relatório, foram observados pacotes cujo tipo Ethernet não correspondia a nenhum protocolo reconhecido pelo *sniffer*, sendo classificados como `"Other"`. Estes pacotes, frequentemente de pequena dimensão (≈59 bytes), resultam de protocolos de controlo internos do *driver* Wi-Fi ou de mecanismos proprietários do fabricante da placa de rede.
4. **Presença de Tráfego Encriptado:** Uma parcela significativa do tráfego **TCP** capturado na interface real opera sobre a porta 443 (HTTPS/TLS). Embora o *sniffer* identifique corretamente o protocolo e as flags **TCP**, o conteúdo do *payload* permanece inacessível, ao contrário do HTTP no CORE onde o *payload* era legível em texto limpo se fosse decodificado.

5.1.6 | Obstáculos Técnicos Encontrados e Resolução

Durante a transposição para a interface real, o grupo deparou-se com os seguintes obstáculos técnicos:

Tabela 3 : Obstáculos técnicos encontrados na transposição para a interface real e respetivas resoluções

Obstáculo	Descrição	Resolução
Permissões de Administrador	A captura em interfaces reais exige obrigatoriamente permissões *root*. No CORE, o emulador já opera com privilégios elevados.	O mecanismo `os.getuid() != 0` já implementado garante a verificação precoce. O utilizador é instruído a executar `sudo python3 main.py`.
Inundação de Tráfego	Na interface real, a captura sem filtro resulta numa avalanche de pacotes que dificulta a análise e sobrecarrega o ecrã.	Utilização de filtros BPF para isolar o tráfego de interesse. O `time.sleep(0.5)` entre pacotes permite a leitura do utilizador antes da chegada do próximo.
Pacotes de Protocolos Desconhecidos	Na rede real surgiram pacotes com tipos Ethernet não convencionais, causando potenciais erros de parsing.	A lógica de *fallback* `"Protocol: Other"` no `protocol.py` garante que o *sniffer* permanece estável, classificando graciosamente pacotes não reconhecidos sem interromper a captura.
Validação de Interface	Caso o utilizador introduza um nome de interface inexistente, o programa poderia falhar ao iniciar a captura.	A verificação `if (iface not in scapy.get_if_list()): iface = ""` no `interface.py` reverte para a interface padrão do sistema em caso de entrada inválida.

5.1.7 | Módulo de Injeção de Tráfego (gerador.py)

Para facilitar a validação do *sniffer* na interface real, foi utilizado o módulo `gerador.py` que permite a geração controlada de pacotes sintéticos de diversos protocolos. Este utilitário é particularmente útil quando se pretende testar a deteção de um protocolo específico sem depender do tráfego espontâneo da rede. O gerador oferece funções para **ICMPv4/v6**, **DNS**, **HTTP**, **DHCPv4/v6**, **QUIC**, entre outros, utilizando a biblioteca **Scapy** para a construção e envio dos pacotes.

5.2 | Limitações da Aplicação

Apesar do elevado grau de funcionalidade alcançado, o grupo reconhece as seguintes limitações na aplicação desenvolvida:

5.2.1 | Protocolos Não Suportados ou Parcialmente Analisados

- **TLS/SSL (Camada de Sessão):** Embora o *sniffer* identifique tráfego **HTTPS** (porta 443) e **QUIC** (UDP 443), a ferramenta não implementa a desencapsulação de *payloads* cifrados. A inspeção profunda (*Deep Packet Inspection*) limita-se a protocolos cujo conteúdo transita em texto limpo (**HTTP, DNS**). Esta é uma limitação inerente ao paradigma de inspeção passiva, dado que a desencriptação exigiria acesso às chaves de sessão **TLS**.
- **IEEE 802.11 (Wi-Fi):** Conforme já documentado previamente, em modo gerido (*Managed Mode* — o modo padrão), o sistema operativo e os controladores da placa de rede abstraem o tráfego rádio, encapsulando-o em cabeçalhos Ethernet II fictícios (*Ethernet Emulation*). Para capturar os cabeçalhos reais de **Wi-Fi** (como *frames* de gestão **Beacon** ou **Probe**), seria necessário colocar a interface em Modo Monitor, o que apresenta dependências de hardware e permissões de sistema complexas. Assim, o grupo optou por focar o desenvolvimento na análise de *frames* Ethernet, garantindo a estabilidade do *sniffer* em ambientes de rede convencionais.
- **Protocolos de Camada 7 Sem Mapeamento de Portas:** A identificação de protocolos de aplicação baseia-se maioritariamente no mapeamento estático de portas (`TCP_MAP` e `UDP_MAP`). Protocolos que operem em portas não convencionais ou dinâmicas (ex.: serviços P2P, aplicações proprietárias) são classificados genericamente como **TCP** ou **UDP**, sem identificação específica do protocolo de aplicação.
- **Análise Limitada de IPv6:** O suporte a **IPv6** contempla a extração de endereços e do *Hop Limit*, bem como a identificação de **ICMPv6** (*Echo Request/ Reply*). Contudo, funcionalidades avançadas como a análise de *Extension Headers* (*Routing, Fragment, Hop-by-Hop*) ou mensagens de *Neighbor Discovery Protocol* (NDP) não foram implementadas em detalhe.

5.2.2 | Limitações de Desempenho e Parsing

- **Processamento Sequencial:** O mecanismo de callback (`live_print`) processa cada pacote de forma sequencial, com uma pausa deliberada (`time.sleep`) entre pacotes. Em redes de alto débito com centenas ou milhares de pacotes por segundo, este modelo pode resultar na perda de pacotes que chegam durante o período de processamento e exibição. Uma solução futura passaria pela implementação de uma *thread* separada para captura e uma fila (*queue*) assíncrona para a exibição.

- **Armazenamento em Memória (`store=True`):** A opção `store=True` no `scapy.sniff()` mantém todos os pacotes capturados em memória RAM. Em sessões de captura prolongadas ou em redes com elevado volume de tráfego, isto pode resultar num consumo de memória crescente e, em casos extremos, no esgotamento de recursos do sistema. Uma alternativa mais robusta consistiria na escrita periódica dos dados em disco (*streaming*) em vez da acumulação em memória.
- **Dependência do `os.system()`:** A utilização de chamadas ao sistema operativo (`os.system`) para operações como limpar o ecrã ou exibir *logs* introduz uma dependência direta do ambiente Linux. Embora esta abordagem seja funcional e eficiente, limita a portabilidade do software para outros sistemas operativos (ex.: Windows, macOS) sem adaptações.

5.2.3 | Funcionalidades Previstas mas Não Implementadas

- **Exportação em Formatos Estruturados:** O sistema de *logging* atual grava os dados em formato de texto simples (`log.txt`). A exportação para formatos como **CSV** ou **JSON**, referidos no enunciado como possibilidades, permitiria a análise posterior em ferramentas externas (ex.: Excel, scripts de análise em Python/ Pandas) e a correlação automatizada de eventos.
- **Métricas de Desempenho de Rede:** Embora o enunciado sugira como opcionais métricas como o débito (*throughput*) e o RTT (*Round Trip Time*), estas não foram implementadas. O cálculo do RTT exigiria a correlação temporal entre pacotes de pedido e resposta (ex.: ICMP Echo Request/Reply), enquanto o débito necessitaria da monitorização contínua do volume de dados por intervalo de tempo.
- **Interface Gráfica (GUI):** O *sniffer* opera exclusivamente em modo terminal. Uma interface gráfica com visualização em tempo real de fluxos de rede, filtragem interativa e gráficos de distribuição de tráfego elevaria significativamente a experiência do utilizador, aproximando a ferramenta de soluções profissionais como o *Wireshark*.
- **Reconstrução de Sessões TCP (*Stream Reassembly*):** O *sniffer* analisa cada pacote de forma individual, sem a capacidade de reconstruir o fluxo de dados completo de uma sessão **TCP**. Esta funcionalidade permitiria, por exemplo, reconstruir uma página HTTP completa a partir dos múltiplos segmentos TCP que a compõem.

6 | Conclusão

O desenvolvimento do **Packet Sniffer** cumpriu o objetivo de criar uma ferramenta modular e funcional para a análise de tráfego de rede. A solução final integra com sucesso a seleção de interfaces, filtragem **BPF**, identificação multinível de protocolos e exportação de dados, suportada por uma arquitetura que garante uma separação clara de responsabilidades.

A validação no emulador CORE e em ambiente real confirmou a eficácia da ferramenta na classificação e *logging* de pacotes. No entanto, foram identificadas limitações em cenários de elevado débito, onde o ciclo síncrono de captura e processamento pode resultar na perda de pacotes e em latência na resposta do sistema.

Como trabalho futuro, a migração para um modelo de captura assíncrona com filas de processamento permitirá aumentar a robustez da ferramenta perante tráfego intenso. Em suma, o **Packet Sniffer** atingiu os objetivos propostos, estabelecendo uma base técnica sólida para futuras evoluções no domínio da monitorização de redes.